

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ імені ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

Кафедра радіоелектронних і комп'ютерних систем

Освітній ступінь «бакалавр»

Галузь знань 12 – Інформаційні технології

(шифр і назва)

Спеціальність 121 – Інженерія програмного забезпечення

(шифр і назва)

«ЗАТВЕРДЖУЮ»

Завідувач кафедри _____

проф. Оленич І.Б.

«__» _____ 2026 р.

ЗАВДАННЯ

НА КВАЛІФІКАЦІЙНУ (БАКАЛАВРСЬКУ) РОБОТУ СТУДЕНТУ

Фіняку Олегу Володимировичу

(прізвище, ім'я, по батькові)

1. Тема роботи: Інформаційно-обчислювальний застосунок для студентів природничих спеціальностей

керівник роботи асистент, кандидат технічних наук Гура В.Т., _____,

затверджені Вченою радою факультету від «__» _____ 202__ року № _____

2. Строк подання студентом роботи _____

3. Вихідні дані до роботи

1. Feature-Sliced Design: Architectural methodology for frontend projects [Електронний ресурс]. – Режим доступу: <https://feature-sliced.design/>

2. React Official Documentation. The library for web and native user interfaces [Електронний ресурс]. – Режим доступу: <https://react.dev/>

3. Firebase Cloud Firestore Documentation [Електронний ресурс]. – Режим доступу: <https://firebase.google.com/docs/firestore>

4. Зміст розрахунково-пояснювальної записки (перелік питань, які потрібно розробити)

1. Аналіз предметної області та огляд технологій.

2. Проектування та розробка програмного забезпечення

3. Тестування, розгортання та оцінка результатів роботи

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

1. Презентація у форматі PowerPoint

6. Консультанти розділів роботи

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		Завдання видав	Завдання прийняв

7. Дата видачі завдання _____

КАЛЕНДАРНИЙ ПЛАН

№ з/п	Назва етапів кваліфікаційної (бакалаврської) роботи	Строк виконання етапів роботи	Примітка
1	Аналіз предметної області та огляд технологій	Грудень 2025-лютий 2026	
2	Проектування та розробка програмного забезпечення	Березень-квітень 2026	
3	Тестування, розгортання та оцінка результатів роботи	Травень 2026	

Студент _____ Фіняк О.В. _____
(підпис) (прізвище та ініціали)

Керівник роботи _____ ас. Гура В.Т. _____
(підпис) (прізвище та ініціали)

ВІДГУК

наукового керівника на бакалаврську роботу
за спеціальністю 121 – Інженерія програмного забезпечення
на тему: Інформаційно-обчислювальний застосунок для студентів природничих спеціальностей
студента денної форми навчання академічної групи ФЕП-41с
Фіняка Олега Володимировича

Критерії	Бали
Ставлення студента до виконуваної ним роботи, ступінь самостійності, ініціативність, систематичність у виконанні роботи (0÷10 б)	
Відповідність роботи завданням, оцінка повноти виконання завдання, ступінь складності роботи (0÷5 б)	
Використання сучасних методів і засобів (0÷5 б)	
Наукова / практична цінність, обґрунтованість висновків (0÷5 б)	
Відповідність вимогам до оформлення та ілюстративність роботи (0÷5 б)	
Особливості, позитивні та негативні риси роботи (текстом, 2–5 речень)	

Науковий керівник

асистент

(науковий ступінь, посада)

(підпис)

Володимир Гура

(ім'я прізвище)

«__» _____ 2026 р.

РЕЦЕНЗІЯ
 на бакалаврську роботу
 за спеціальністю 121 – Інженерія програмного забезпечення
 на тему: Інформаційно-обчислювальний застосунок для студентів природничих спеціальностей
 студента денної форми навчання академічної групи ФЕП-41с
 Фіняка Олега Володимировича

Критерії	Бали
Ступінь актуальності роботи, відповідність змісту роботи обраній темі (0÷4 б)	
Якість та повнота аналітичного огляду літератури, що базується на ґрунтовному огляді провідних вітчизняних та зарубіжних публікацій; наявність відповідних цитувань (0÷4 б)	
Обґрунтованість застосування методик та інструментів (0÷2 б)	
Представлення та аналіз результатів (0÷4 б)	
Наукова новизна / практична цінність (0÷2 б)	
Цілісність та аргументованість висновків (0÷2 б)	
Якість ілюстративного матеріалу, відповідність роботи вимогам до оформлення (0÷2 б)	
Обсяг і структура роботи	сторінок – розділів – рисунків – таблиць – першоджерел – додатків –
Помилки, недоліки, позитивні сторони роботи; зауваження та пропозиції (за наявності)	
Загальний висновок щодо відповідності бакалаврської роботи вимогам («повністю відповідає», «загалом відповідає», «повністю не відповідає») та оцінка бакалаврської роботи	

Рецензент

(науковий ступінь, посада)

(підпис)

(ім'я прізвище)

«__» _____ 2026 р.

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра радіоелектронних і комп'ютерних систем

Допустити до захисту

Завідувач кафедри

_____ проф. Оленич І.Б.
(підпис) (ПІБ)

«___» _____ 2026 р.

Кваліфікаційна робота

Бакалавр
(освітній ступінь)

ІНФОРМАЦІЙНО-ОБЧИСЛЮВАЛЬНИЙ ЗАСТОСУНОК ДЛЯ СТУДЕНТІВ ПРИРОДНИЧИХ СПЕЦІАЛЬНОСТЕЙ

Виконав:

студент групи ФЕП-41с
спеціальності 121 – Інженерія
програмного забезпечення

_____ Фіняк О.В.
(підпис) (ПІБ)

Науковий керівник:

_____ ас. Гура В.Т.
(підпис) (ПІБ)

«___» _____ 2026 р.

Рецензент:

_____ (підпис) (ПІБ)

«___» _____ 2026 р.

Львів 2026

АНОТАЦІЯ

У роботі розроблено інформаційну систему «SciLearn», яка спрощує навчання для студентів природничих спеціальностей, автоматизуючи складні наукові розрахунки та пошук навчальних матеріалів. Проєкт побудовано як сучасний вебзастосунок, що вирізняється зручною навігацією, швидкою роботою та модульною архітектурою для легкого розвитку в майбутньому. Система містить інтерактивний калькулятор, що автоматично перевіряє фізичну коректність введених даних та якісно відображає складні формули, усуваючи ризик арифметичних помилок. Платформа враховує індивідуальні інтереси користувачів, пропонуючи найбільш актуальні матеріали та задачі на основі їхньої активності. Ключовою особливістю є інтелектуальний помічник, що відповідає на запитання на основі університетської програми та продовжує працювати навіть без доступу до інтернету завдяки розумному зберіганню знань. Тестування підтвердило високу точність обчислень, надійність системи в умовах нестабільного зв'язку та позитивний вплив на ефективність самостійної роботи студентів. Результати роботи готові до впровадження у навчальний процес як надійний інструмент для підтримки вищої освіти.

ABSTRACT

This paper presents the "SciLearn" information system, designed to simplify the learning process for natural science students by automating complex scientific calculations and organizing study materials. The project is built as a modern web application, characterized by intuitive navigation, high performance, and a modular architecture for easy future development. The system includes an interactive calculator that automatically checks the physical validity of data and correctly displays complex formulas, eliminating the risk of arithmetic errors. The platform takes into account individual user interests, suggesting the most relevant materials and tasks based on previous activity. A key feature is an intelligent assistant that answers questions based

on the university curriculum and continues to function even without internet access due to smart local knowledge caching. Testing confirmed the high accuracy of calculations, system reliability in conditions of unstable connectivity, and a positive impact on the efficiency of students' independent work. The results of the work are ready for implementation in the educational process as a reliable tool for supporting higher education.

ЗМІСТ

ВСТУП.....	5
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД ТЕХНОЛОГІЙ	7
1.1. Аналіз сучасних методів та засобів розробки інтерактивних освітніх систем	7
1.2. Огляд існуючих програмних платформ та виділення невирішених проблем.....	9
1.3. Аналіз архітектурних підходів до інтеграції штучного інтелекту у веб-застосунки	13
1.4. Формулювання науково-технічної задачі та обґрунтування мети роботи	16
РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ СИСТЕМИ «SCILEARN»	19
2.1. Формулювання вимог до програмного проєкту.....	19
2.1.1. Вимоги до клієнтської частини (front-end) та інтерфейсу користувача.....	21
2.1.2. Вимоги до серверної інфраструктури (back-end) та збереження даних	25
2.2. Проєктування архітектури системи	28
2.2.1. Загальна структурна схема проєкту за методологією Feature-Sliced Design.....	30
2.2.2. Розробка UML-діаграм та схем баз даних (Formula, Theory, Problem)	34
2.3. Обґрунтування вибору засобів і методів розробки	37
2.4. Програмна реалізація клієнтської частини та обчислювального модуля	40
2.5. Програмна реалізація системи рекомендацій на основі колаборативної фільтрації.....	43
2.6. Інтеграція механізму Graph/Keyword RAG для локального пошуку та генерації відповідей AI-асистентом	46
РОЗДІЛ 3. ТЕСТУВАННЯ, РОЗГОРТАННЯ ТА ОЦІНКА РЕЗУЛЬТАТІВ РОБОТИ ...	49
3.1. Тестування програмного забезпечення.....	49
3.1.1. Розробка тест-кейсів та модульне тестування алгоритмічного ядра.....	52
3.1.2. Ступінь покриття вимог до проєкту та коду (Characterization testing)	54
3.2. Опис роботи та застосування проєкту.....	56
3.2.1. Покрокова інструкція з розгортання програмного продукту	60
3.2.2. Керівництво користувача веб-застосунку	62
3.3. Аналіз та узагальнення результатів роботи системи	66
ВИСНОВКИ	70
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	72
ДОДАТКИ.....	74
Додаток А. Лістинг коду формул у physics.ts	74
Додаток Б. Програмна реалізація обчислювального ядра та хука калькулятора.....	75
Додаток В. Взаємодія з хмарною базою Cloud Firestore.....	77
Додаток Г. Реалізація ІІІ-асистента та механізму RAG	81

ВСТУП

На сучасному етапі розвитку вищої освіти спостерігається стрімка тенденція до цифровізації навчального процесу. Перехід до сучасних форм навчання вимагає від закладів освіти розробки та впровадження спеціалізованих платформ, здатних забезпечити студентам швидкий і зручний доступ до методичного забезпечення. Особливої уваги в цьому контексті потребують студенти природничих спеціальностей (фізичного, хімічного, біологічного напрямків). Специфіка їхнього навчання полягає в необхідності опрацювання великих обсягів спеціалізованої літератури, проведення експериментальних досліджень та виконання складних математичних розрахунків. На відміну від гуманітарних дисциплін, природничі науки вимагають постійної роботи з формулами, константами, табличними даними та алгоритмами обчислень.

З інженерної точки зору, сьогодні існує суттєвий розрив між потребами студентів-природничиків та наявними програмними засобами. Більшість університетських систем управління навчанням (LMS), таких як Moodle або Google Classroom, є універсальними інструментами, які не мають специфічного функціоналу для підтримки безпосередньої науково-розрахункової роботи. Як наслідок, навчальний процес супроводжується децентралізацією інформації: методичні вказівки розміщуються у хмарних сховищах, довідкові дані у підручниках, а розрахунки виконуються сторонніми програмами. Така фрагментація не лише призводить до втрати часу, але й підвищує когнітивне навантаження та ймовірність помилок під час виконання лабораторних робіт. Відповідно, створення спеціалізованої веб-орієнтованої інформаційної системи, яка консолідує структуровану бібліотеку методичних матеріалів та інтерактивний інструментарій для автоматизації обчислень, є актуальним завданням сучасної інженерії програмного забезпечення.

Додатковим інженерним викликом є інтеграція сучасних технологій штучного інтелекту. Традиційне впровадження інтелектуальних асистентів у вебзастосунки часто супроводжується галюцинаціями мовних моделей, що є неприпустимим у точних науках. Розв'язання цієї проблеми вимагає архітектурних рішень, здатних обмежити штучний інтелект контекстом закритої

бази знань (методичних матеріалів та формул), забезпечуючи при цьому стійкість платформи до автономної роботи без розгортання ресурсомісткої бекенд-інфраструктури.

Об'єктом дослідження є процес інформаційного забезпечення та підтримки навчальної діяльності студентів природничих спеціальностей.

Предметом дослідження виступають методи, алгоритми та архітектурні рішення розробки веб-орієнтованих систем для структурування контенту, автоматизації обчислень та інтеграції локальних механізмів генеративного пошуку (Graph/Keyword RAG).

Метою роботи є проєктування та розробка програмного забезпечення спеціалізованої вебплатформи «SciLearn», що забезпечить консолідацію навчальних матеріалів, автоматизацію рутинних обчислювальних процесів та персоналізацію контенту за допомогою штучного інтелекту, обмеженого предметною областю природничих факультетів.

Наукова новизна та практичне значення. У роботі запропоновано інженерний підхід до створення інтегрованого освітнього середовища, яке відходить від традиційної структури електронних бібліотек на користь поєднання статичного контенту (методичних та теоретичних матеріалів) з динамічними інструментами (інтерактивними калькуляторами), безпосередньо адаптованими до контексту задачі. Практичне значення полягає у розробці системи, що вирішує проблему фрагментації навчальних матеріалів. Запропонована платформа забезпечує жорстку структурування знань, зменшує рутинне математичне навантаження завдяки вбудованим алгоритмам розрахунків і гарантує безперебійний доступ до матеріалів з будь-яких пристроїв без встановлення додаткового програмного забезпечення.

РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ОГЛЯД ТЕХНОЛОГІЙ

1.1. Аналіз сучасних методів та засобів розробки інтерактивних освітніх систем

Сучасна інженерія програмного забезпечення пропонує широкий спектр методологій та архітурних патернів для проєктування складних інформаційних систем освітнього призначення. Процес цифровізації методичного забезпечення у вищих навчальних закладах вимагає переходу від статичного представлення інформації до створення динамічних інтерактивних середовищ, що здатні адаптуватися під специфіку конкретних предметних областей, зокрема природничих дисциплін. Традиційно розробка таких систем базувалася на монолітних архітектурах, де рендеринг інтерфейсу та обробка бізнес-логіки відбувалися виключно на стороні сервера. Проте впровадження сучасних вебтехнологій змістило парадигму проєктування у бік односторінкових застосунків (Single Page Applications, SPA), які переносять завдання формування інтерфейсу безпосередньо у браузер користувача. Такий підхід дозволяє кардинально знизити навантаження на серверну інфраструктуру, мінімізувати мережеві затримки та забезпечити плавність взаємодії, що є критично важливим для утримання уваги користувача під час тривалої роботи з навчальними матеріалами.

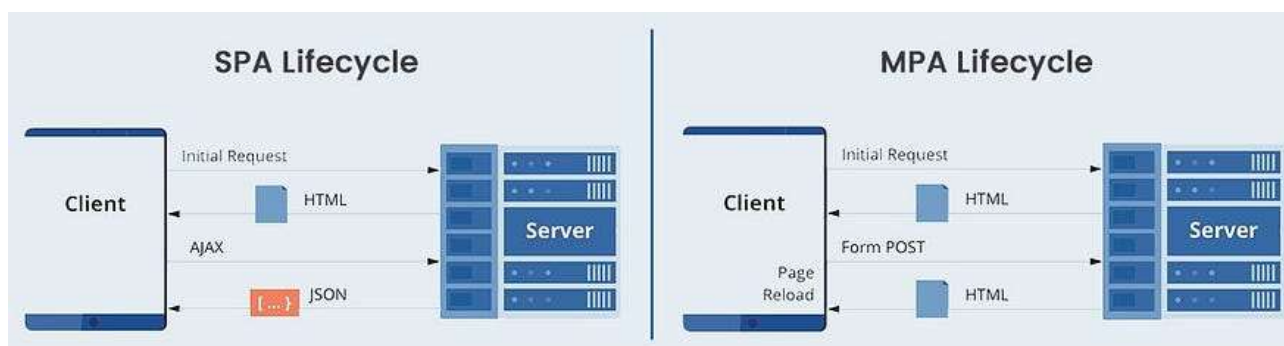


Рисунок 1.1.Схема взаємодії між клієнтом та сервером у архітектурних моделях MPA та SPA

Основним методом побудови сучасних клієнтських вебсистем є компонентно-орієнтований підхід. Він передбачає декомпозицію складного

інтерфейсу користувача на набір ізольованих, незалежних та перевикористовуваних модулів, кожен з яких відповідає за власну логіку та відображення даних. У контексті інженерії освітнього програмного забезпечення це дає змогу розділити інтерфейс на незалежні складові, такі як модулі теоретичного контенту, інтерактивні розрахункові калькулятори та віджети інтелектуальної підтримки. Використання сучасних компонентних фреймворків дозволяє реалізувати чітку декларативну модель управління станом застосунку, що значно спрощує процес розробки, підтримки та подальшого масштабування програмного продукту. Оптимізація доставки таких компонентів клієнту досягається за допомогою інструментів автоматизованої збірки, які виконують розділення коду (code splitting) на окремі чанки та застосовують механізми лінивого завантаження (lazy loading). Це мінімізує обсяг початкового JavaScript-коду, що завантажується користувачем, і забезпечує високу швидкість відгуку платформи навіть на мобільних пристроях.

Важливою складовою сучасних методів розробки є інтеграція спеціалізованих клієнтських бібліотек для обробки нетекстової інформації. Для освітніх систем природничого спрямування критично важливою є підтримка якісного рендерингу складних математичних та хімічних виразів, записаних у форматі LaTeX. Використання швидких систем обчислення та візуалізації на стороні клієнта дозволяє відмовитися від генерації статичних зображень формул на сервері, забезпечуючи динамічне масштабування та відповідність вимогам цифрової ергономіки. Разом із тим сучасні підходи до проектування клієнтської логіки орієнтовані на забезпечення автономності застосунків (offline-first). Це досягається шляхом впровадження механізмів локального кешування, збереження стану в пам'яті браузера та використання швидких алгоритмів пошуку безпосередньо над клієнтськими структурами даних. Таким чином, сучасні засоби розробки дозволяють створити стійку та незалежну програмну систему, яка здатна консолідувати розрізнені методичні матеріали та надавати інструменти для наукових розрахунків в єдиному інтегрованому цифровому середовищі.

1.2. Огляд існуючих програмних платформ та виділення невирішених проблем

Аналіз сучасного стану цифровізації вищої освіти дозволяє класифікувати наявні програмні інструменти, якими користуються студенти природничих спеціальностей, на кілька основних категорій. Першу та найбільш масову групу складають універсальні системи управління навчанням (Learning Management Systems, LMS), серед яких найбільш поширеними є Moodle та Google Classroom. З інженерної точки зору, головною архітектурною перевагою таких систем є їхня багатофункціональність та здатність забезпечувати адміністрування курсів, контроль успішності та збір звітів у межах усього навчального закладу. Проте саме концепція універсальності стає їхнім ключовим недоліком при роботі зі специфічним контентом природничих дисциплін. Ці платформи проектувалися як узагальнені репозиторії файлів та текстових завдань, тому вони повністю позбавлені вбудованого інструментарію для інтерактивної взаємодії з фізичними чи хімічними процесами. Вони не мають засобів для динамічного рендерингу математичних розмірностей, побудови графіків та проведення розрахунків у реальному часі, що змушує розробників та викладачів інтегрувати сторонні модулі, які часто порушують цілісність інтерфейсу та знижують загальну швидкодію системи. Окрім цього, монолітна структура традиційних LMS часто призводить до перевантаження карти навігації, коли шлях користувача до конкретного методичного матеріалу вимагає значної кількості переходів по меню, що суттєво погіршує показники цифрової ергономіки.

Другу категорію засобів складають несистематизовані цифрові ресурси, до яких відносяться корпоративні або публічні хмарні сховища та канали комунікації у месенджерах. За умов відсутності спеціалізованого програмного забезпечення цей підхід стає основним каналом передачі методичних матеріалів у формі статичних документів. Проте використання таких інструментів створює проблему високого інформаційного шуму та унеможливорює організацію ефективного пошуку інформації. Студент змушений працювати зі скан-копіями або фотографіями низької якості, які не підтримують повнотекстовий аналіз та не адаптуються під екрани мобільних пристроїв. Більше того, у таких

середовищах повністю відсутній контроль версійності даних, через що виникають ризики використання застарілих або помилкових методичних вказівок. Головним архітектурним недоліком цієї моделі є повна ізолюваність навчального тексту від будь-яких обчислювальних засобів, що перетворює процес підготовки до практичних чи лабораторних робіт на хаотичне перемикавання між різнорідними програмами.

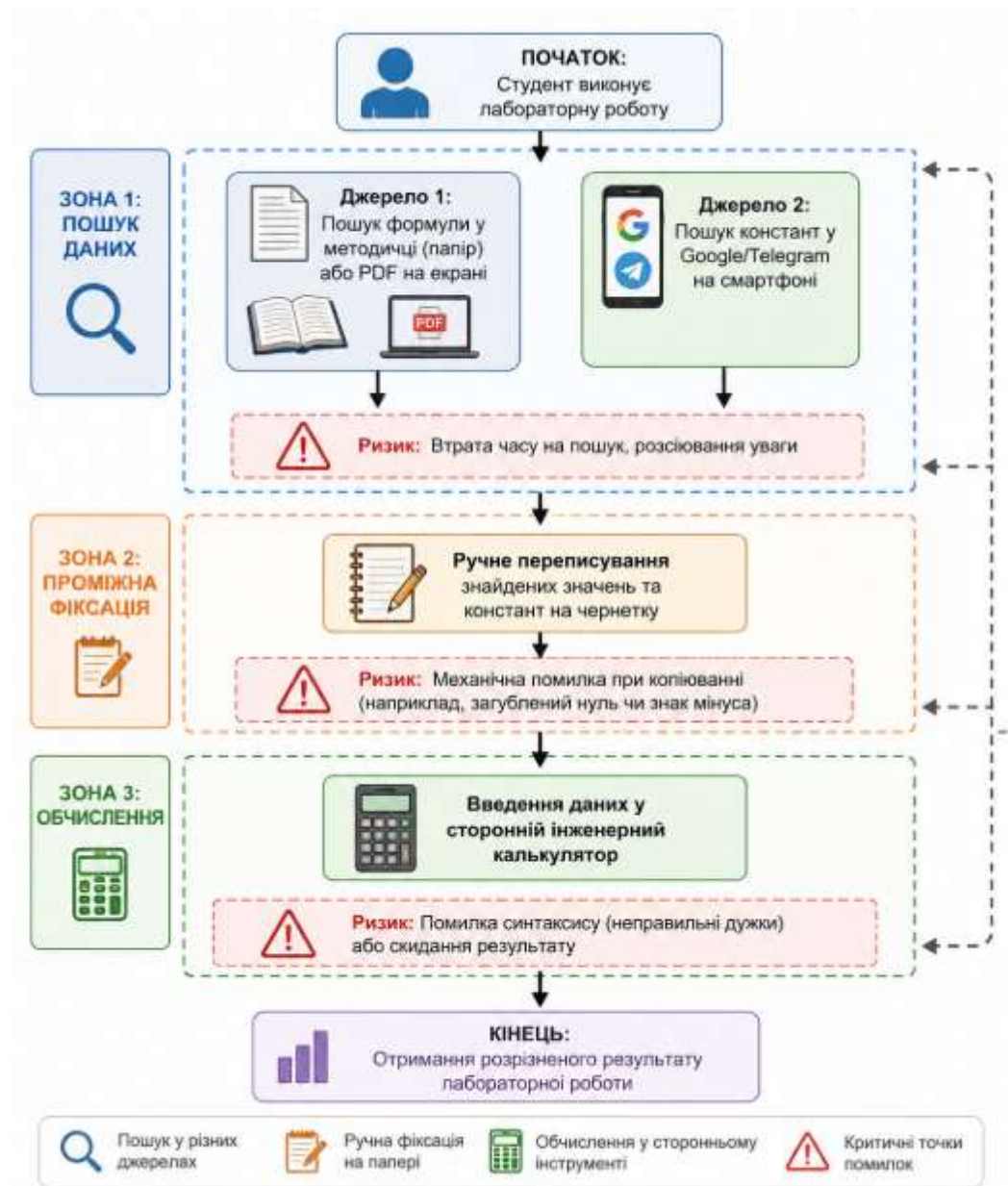


Рисунок 1.2. Схема фрагментованого процесу взаємодії студента з розрізненими джерелами даних під час виконання наукових розрахунків.

Окреме місце в освітньому процесі займають високоспеціалізовані обчислювальні платформи та інтелектуальні довідкові рушії, найяскравішим

представником яких є WolframAlpha. Ці системи володіють потужним математичним апаратом і здатні розв'язувати складні аналітичні задачі. Проте з погляду інженерії програмного забезпечення вони функціонують за принципом ізольованого обчислювального ядра, яке є абсолютно відірваним від контексту конкретної навчальної програми. Система не володіє інформацією про те, яку саме лабораторну роботу чи тему наразі вивчає студент, і вимагає від нього самостійного та синтаксично точного формування запитів. Такі інструменти орієнтовані на отримання миттєвого результату, а не на дидактичний супровід, тому вони не пропонують покрокових методичних роз'яснень, адаптованих під рівень знань користувача. Крім того, робота подібних платформ повністю залежить від безперервного зв'язку із закритими серверними API, що створює критичну точку відмови у разі відсутності швидкого інтернет-з'єднання.

Огляд існуючих рішень дозволяє чітко сформулювати ключову невирішену проблему: наявність глибокого розриву між теоретичним базисом, методичними інструкціями та обчислювальним апаратом. Традиційний ланцюжок дій студента під час розрахунку параметрів лабораторної роботи складається з ручного пошуку формули в текстовому документі, копіювання табличних констант із сторонніх джерел та введення цих даних в інженерний калькулятор. Цей процес супроводжується значною цифровою фрагментацією навчального середовища, що призводить до високого когнітивного навантаження та виникнення механічних помилок при перенесенні цифр чи перетворенні одиниць вимірювання. Сучасні інформаційні системи не пропонують механізмів персоналізованого підбору контенту на основі попередніх взаємодій користувача, що описується в теорії сучасного інформаційного пошуку та рекомендаційних систем як проблема «холодного старту» або низької релевантності вибірки. Таким чином, розробка інтегрованого програмного рішення, яке об'єднує статичний методичний матеріал із динамічним обчислювальним модулем та контекстно-залежним штучним інтелектом в межах єдиної SPA-архітектури, є актуальною інженерною задачею,

що дозволить ліквідувати фрагментацію даних та автоматизувати рутинні процеси.

Інженерно-технічні критерії	Moodle / Google Classroom	Месенджери (Telegram, Viber) та хмарні папки	WolframAlpha	Система SciLearn
Наявність контекстної інтеграції обчислень	Відсутня	Відсутня	Є	Є
Вбудована підтримка рендерингу наукового контенту (LaTeX)	Часткова	Відсутня / Обмежена	Є	Є
Наявність системи персоналізованих рекомендацій	Відсутня	Відсутня	Відсутня	Є
Контекстно-залежний ШІ-помічник	Відсутня	Відсутня	Є	Є
Можливість повноцінного функціонування в офлайн-режимі	Обмежена	Часткова	Відсутня	Є
Ергономічність навігації	Низька	Дуже низька	Середня	Висока

Таблиця 1.1 Порівняльний інженерно-технічний аналіз існуючих програмних платформ та системи SciLearn.

1.3. Аналіз архітектурних підходів до інтеграції штучного інтелекту у веб-застосунки

Інтеграція технологій штучного інтелекту, зокрема великих мовних моделей, у сучасні інформаційні системи вимагає глибокого аналізу архітектурних патернів та методів організації потоків даних. Класичний підхід до вирішення цієї інженерної задачі базується на використанні серверних інфраструктур, де вебзастосунок виступає виключно в ролі тонкого клієнта, що перенаправляє запити користувача на бекенд-платформу. У таких архітектурах для забезпечення релевантності відповідей штучного інтелекту традиційно застосовують концепцію генерації з розширенням пошуку (Retrieval-Augmented Generation, RAG). Реалізація стандартного RAG-конвеєра передбачає розгортання та підтримку спеціалізованих векторних баз даних, інструментів для генерації ембедінгів та проміжних оркестраторів контексту. Процес обробки запиту у такій системі включає його перетворення у високорозмірний вектор, виконання семантичного пошуку за метрикою косинусної схожості серед масивів неструктурованих даних, формування промпту та передачу його до хмарного API мовної моделі. Попри високу ефективність при роботі з гігантськими обсягами інформації, цей підхід створює значне інфраструктурне та фінансове навантаження, збільжує затримку відповіді (latency) через багатоетапні мережеві транзакції та утворює критичну точку відмови у разі збою серверної частини системи.



Рисунок 1.3 Структурна схема класичного серверного RAG-конвеєра з використанням хмарної векторної бази даних.

Особливу складність при проєктуванні інтелектуальних освітніх платформ для точних та природничих наук становить проблема фактологічної достовірності інформації. Пряма взаємодія з базовими мовними моделями часто супроводжується галюцинаціями або видачею застарілих даних, що є неприпустимим у навчальному процесі, де точність формул та констант має вирішальне значення. Застосування вищезгаданого семантичного векторного пошуку не завжди здатне вирішити цю проблему у межах конкретної навчальної дисципліни. Оскільки вектори відображають загальну семантичну близькість слів, система може вилучати контекстуальні блоки, які містять схожі терміни, але належать до абсолютно інших розділів або рівнів складності. Наприклад, запит студента щодо розрахунку сили може помилково залучити матеріали з гідродинаміки замість класичної механіки. Для подолання цього обмеження в інженерії програмного забезпечення активно розвивається підхід, заснований на поєднанні генеративних моделей із графами знань (Graph RAG). Проєктування системи навколо структурованого графу, де вузлами є конкретні теми, формули та задачі, а ребрами – явні логічні зв'язки між ними, дозволяє з високою точністю вилучати саме той контекст, який відповідає затвердженій ієрархії навчального курсу, повністю виключаючи нерелевантні дані.

У сучасних умовах розвитку вебтехнологій та підвищення обчислювальної потужності клієнтських пристроїв перспективним архітектурним рішенням є перенесення логіки RAG-конвеєра безпосередньо на сторону фронтенд-частини застосунку. Для систем із відносно невеликим, але жорстко структурованим обсягом знань, таких як окремі навчальні курси чи довідники, відсутня інженерна необхідність у розгортанні важких хмарних векторних сховищ. Логічна модель даних, що описує взаємозв'язки між теоретичними статтями, математичними формулами та покроковими розв'язаннями задач, може бути повністю індексована в оперативній пам'яті браузера. Використання клієнтських алгоритмів нечіткого пошуку та нормалізації тексту дозволяє виконувати

миттєву валідацію намірів користувача (intent detection) та здійснювати трасування локального графу знань без жодного мережевого запиту. Сформований таким чином контекст передається напряму до полегшених хмарних моделей через оптимізовані REST API рушії. Таке архітектурне рішення забезпечує максимальну швидкість SPA-платформи, знижує витрати на підтримку інфраструктури та дозволяє реалізувати стійкий механізм офлайн-резервування (offline fallback). У разі відсутності інтернету або перевищення квот доступу до API, система не припиняє роботу, а переходить на алгоритм локального синтезу відповідей, формуючи роз'яснення безпосередньо з текстових вузлів підключеного графу знань.

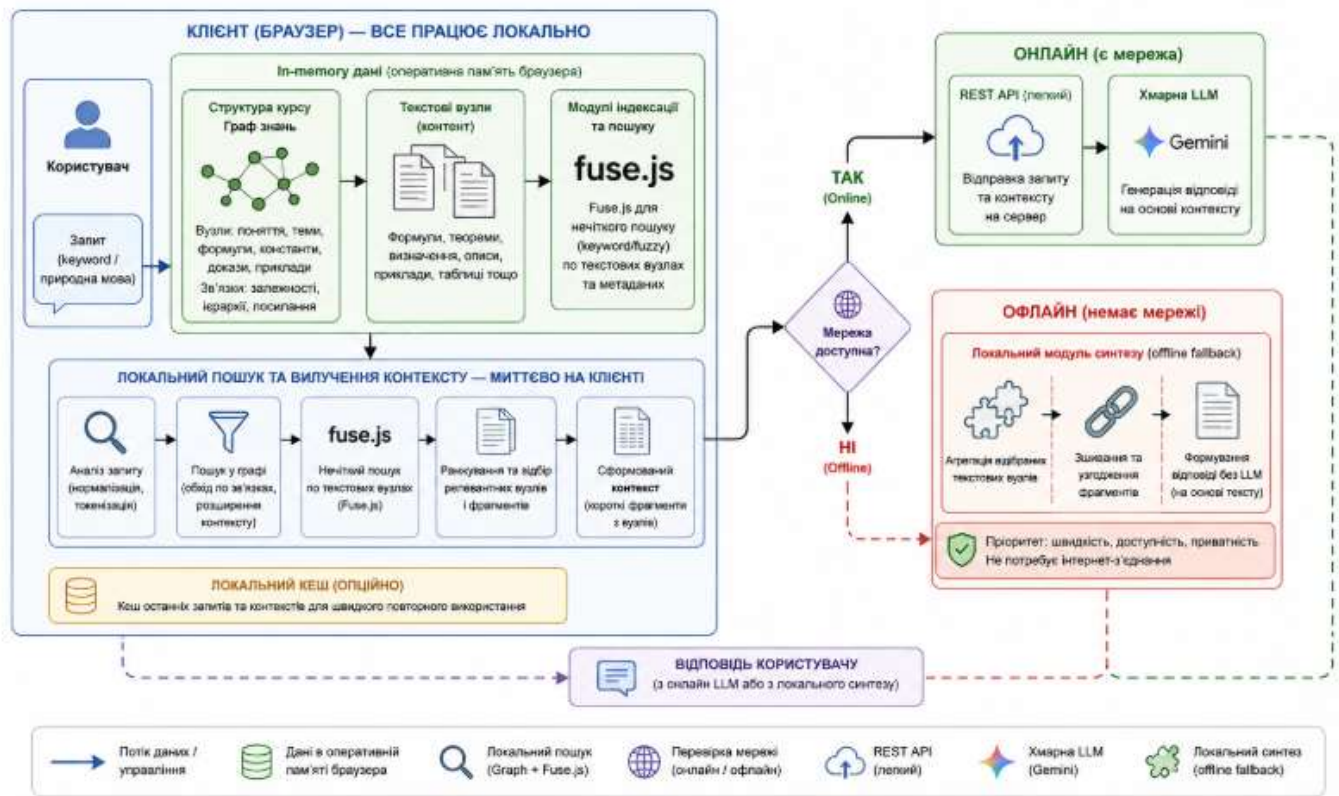


Рисунок 1.4 Архітектурна схема клієнтського Graph/Keyword RAG-конвеєра системи SciLearn з механізмом offline fallback.

1.4. Формулювання науково-технічної задачі та обґрунтування мети роботи

Аналіз предметної області, існуючих програмних рішень та архітектурних підходів до побудови інтелектуальних систем засвідчує, що створення ефективного інструменту для студентів природничих спеціальностей вимагає розв'язання комплексної інженерної проблеми. Ця проблема полягає у ліквідації інформаційної фрагментації навчального середовища шляхом проектування єдиної, відмовостійкої та швидкодіючої архітектури вебзастосунку, яка консолідує статичні методичні матеріали, динамічний обчислювальний модуль та контекстно-залежний штучний інтелект. Таким чином, науково-технічна задача кваліфікаційної роботи полягає у розробці та дослідженні методів проектування клієнт-орієнтованого програмного забезпечення, що забезпечує жорстку структурування знань та автоматизацію обчислювальних процесів безпосередньо у браузері користувача.

Обґрунтування мети роботи безпосередньо впливає з необхідності усунення розриву між теоретичним базисом дисциплін та їхнім практично-розрахунковим апаратом. Впровадження пропонованої системи «SciLearn» дозволить кардинально знизити когнітивне навантаження на студентів, мінімізувати кількість механічних помилок при ручному перенесенні даних та забезпечити безперервний доступ до верифікованої бази знань з будь-яких пристроїв без необхідності розгортання складних серверних рішень. Практична значущість проєкту підсилюється створенням архітектури, здатної функціонувати в автономному режимі, що робить програмний продукт стійким до відмов мережевої інфраструктури.

Для реалізації поставленої мети загальна науково-технічна задача декомпозується на низку послідовних інженерних підзадач, які доцільно розглядати як окремі кроки проектування та розробки системи.

Перша підзадача полягає у дослідженні та формалізації вимог до системи, а також у проектуванні логічної моделі даних, яка повинна гнучко описувати ієрархічну структуру навчального матеріалу природничих дисциплін. Це

передбачає розробку типізованих схем для формул, теоретичних статей та практичних завдань, що дозволить забезпечити між ними явні зв'язки для подальшої побудови графових структур знань.

Друга підзадача охоплює проектування архітектури односторінкового застосунку на основі методології Feature-Sliced Design, що вимагає розробки чіткої тришарової структури та правил односпрямованих залежностей модулів. Таке рішення дозволить повністю ізолювати бізнес-логіку від компонентів представлення та забезпечити високу масштабованість і тестованість вихідного коду.

Третя підзадача спрямована на програмну реалізацію математичного обчислювального модуля, здатного інтерпретувати вхідні змінні та динамічно розраховувати невідомі параметри безпосередньо на стороні клієнта. Цей модуль має безперешкодно інтегруватися з ергономічними засобами рендерингу LaTeX-виразів для забезпечення візуальної відповідності стандартам наукової документації.

Четверта підзадача полягає у розробці та впровадженні клієнтського рушія персоналізованих рекомендацій на основі алгоритмів колаборативної фільтрації. Інженерне завдання тут зводиться до побудови векторів користувацьких взаємодій та обчислення косинусної схожості між профілями в умовах обмежених обчислювальних ресурсів браузера, з обов'язковим проектуванням ефективного fallback-алгоритму для вирішення проблеми «холодного старту».

П'ята підзадача включає проектування та інтеграцію інтелектуального асистента за допомогою механізму Graph/Keyword RAG. Це вимагає розробки алгоритму автоматичного виведення графу знань із структури курсу та створення конвеєра обробки запитів на основі ланцюга респондентів (Responder Chain), який виконуватиме локальну валідацію намірів користувача, вилучення контексту та звернення до хмарних моделей, з обов'язковим моделюванням механізму автономного офлайн-синтезу відповідей.

Шоста підзадача передбачає верифікацію та оцінку надійності розробленого програмного забезпечення за допомогою написання розширеної

бази модульних та характеристичних тестів. Такий крок дозволить зафіксувати очікувану поведінку алгоритмічного ядра, перевірити стійкість обчислювальних модулів та гарантувати працездатність системи в ізольованих умовах без доступу до мережі та зовнішніх хмарних інфраструктур.

РОЗДІЛ 2. ПРОЄКТУВАННЯ ТА РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ СИСТЕМИ «SCILEARN»

2.1. Формулювання вимог до програмного проєкту

Процес інженерії вимог є фундаментальним етапом життєвого циклу розробки програмного забезпечення, оскільки саме на основі сформованої специфікації (Software Requirements Specification) визначається майбутній архітектурний вигляд системи та критерії її успішної валідації [1]. У контексті проєктування інформаційно-обчислювального застосунку «SciLearn» етап формалізації вимог вимагав детального аналізу потреб кінцевих користувачів студентів природничих факультетів, а також урахування технічних обмежень сучасного вебсередовища. Головною метою цього процесу стало перетворення концептуальних дидактичних та розрахункових задач на чіткі інженерні орієнтири, які б дозволили усунути виявлену проблему цифрової фрагментації та обчислювальних помилок у навчальному процесі. Специфікація вимог до системи будувалася навколо забезпечення трьох ключових векторів: швидкого доступу до структурованої бази знань, автоматизації математичного апарату безпосередньо у браузері та надання контекстно-залежної інтелектуальної підтримки.

З урахуванням стандартів програмної інженерії, усі виявлені вимоги до системи «SciLearn» були класифіковані на функціональні, які описують безпосередню поведінку додатку та операції над даними, та нефункціональні, що визначають якісні характеристики продукту, такі як швидкодія, надійність, безпека та відмовостійкість. Особливістю інженерного аналізу для даного проєкту стало те, що архітектурні обмеження зокрема, орієнтація на клієнт-орієнтований SPA-підхід та забезпечення офлайн-стійкості виступили наскрізними чинниками, які суттєво вплинули на формування обох груп вимог. Формалізована структура вимог дозволяє чітко розмежувати відповідальність між окремими шарами програмної системи, що створює передумови для використання сучасних патернів модульного проєктування та полегшує

написання характеристичних тестів для верифікації коду на наступних етапах розробки.



Рисунок 2.1 Схема концептуальної структури вимог та зв'язків між функціональними модулями платформи SciLearn

2.1.1. Вимоги до клієнтської частини (front-end) та інтерфейсу користувача

Клієнтська частина інформаційно-обчислювальної системи «SciLearn» повинна проєктуватися як високопродуктивний односторінковий застосунок (Single Page Application, SPA), що є базовим стандартом для створення сучасних інтерактивних вебсередовищ із насиченою логікою на стороні користувача. Забезпечення плавності інтерфейсу, миттєвого відгуку на дії та безперервної навігації вимагає впровадження декларативної маршрутизації на стороні клієнта за допомогою бібліотеки React Router DOM. Це дозволяє повністю ліквідувати потребу в перезавантаженні сторінок браузера та реалізувати надійне збереження стану екранів під час переходів користувача. Основна вимога до ієрархії відображення полягає в реалізації чіткої трирівневої структури навігації, яка включає головну сторінку з персоналізованими рекомендаціями матеріалів, екрани категорій конкретних природничих дисциплін, а саме фізики, хімії та біології, а також деталізовані сторінки окремих тем, теоретичних статей та формульних калькуляторів.

Важливою нефункціональною вимогою до інтерфейсу є його повна адаптивність (responsiveness) та строга відповідність принципам мобільної ергономіки. Оскільки кінцеві користувачі повинні мати стабільний та зручний доступ до бази знань безпосередньо під час виконання лабораторних чи практичних робіт в аудиторіях, фронтенд-частина має коректно підлаштовувати макет під екрани будь-яких мобільних пристроїв, смартфонів та планшетів. Вебплатформа повинна надавати механізм безшовного перемикання між світлою та темною темами оформлення, адаптуючи контрастність та колірну палітру без мережевих запитів до сервера. Також обов'язковою інженерною вимогою є повна інтернаціоналізація (i18n) інтерфейсу користувача на основі інструментів бібліотеки i18next. Це забезпечує синхронну підтримку української та англійської локалізацій для всіх навігаційних елементів, контекстних меню, кнопок керування та текстових блоків системи.

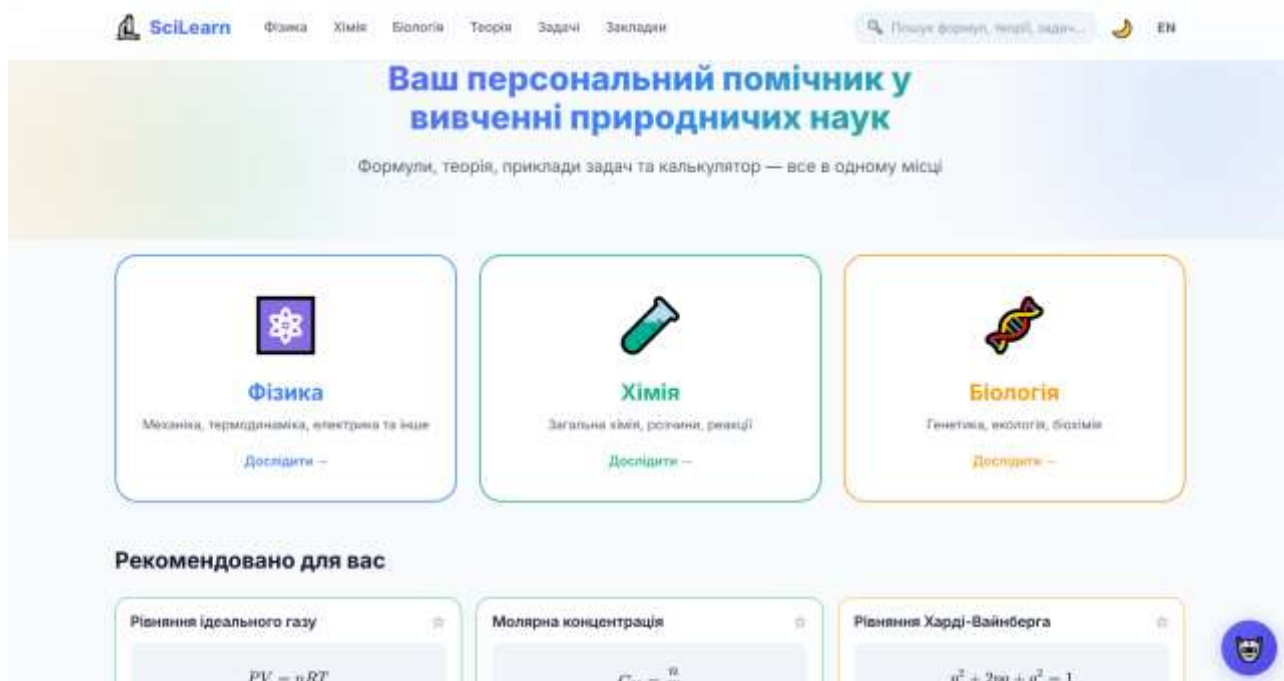


Рисунок 2.2 Інтерфейс головної сторінки застосунку SciLearn з блоком персоналізованих рекомендацій та вибором дисциплін

Особливе місце у специфікації клієнтської частини посідає рендеринг специфічного наукового контенту точних дисциплін. Інтерфейс повинен забезпечувати якісне, швидке та векторне відображення математичних, фізичних та хімічних виразів, записаних у стандартному академічному форматі LaTeX, що реалізується через інтеграцію клієнтської бібліотеки KaTeX. З метою повного усунення інформаційної фрагментації, екран деталізації кожної з 78 формул повинен містити інтерактивну обчислювальну панель, вхідні поля якої динамічно зв'язуються зі змінними параметрами саме цієї формули. Користувач повинен мати можливість вводити числові значення для відомих параметрів і в один клік отримувати автоматичний розрахунок шуканої величини з правильним виведенням одиниць вимірювання, таких як Паскалі, молі чи кілограми, що виключає появу арифметичних помилок.

The screenshot displays the SciLearn website's formula calculator interface. At the top, there is a navigation bar with links for 'Фізика', 'Хімія', 'Біологія', 'Теорія', 'Задані', and 'Заклади'. A search bar on the right contains the text 'Пошук формул, теорій, задач...'. The main content area is titled 'Обчислити' (Calculate) and features the formula $PV = nRT$. Below the formula, there are four input fields with labels and units: 'n' (Кількість речовини, моль [mol]), 'R' (Газова стала, Дж/(моль·К)), 'T' (Температура, К [K]), and 'V' (Об'єм, м³ [m³]). Each field has a 'Введіть значення' (Enter value) placeholder. A blue 'Обчислити' (Calculate) button is positioned at the bottom of the form. A small circular logo is visible in the bottom right corner of the interface.

Рисунок 2.3 Інтерфейс сторінки деталізації формули «Рівняння ідеального газу» з інтегрованим калькулятором змінних

Для забезпечення швидкої навігації та гнучкого доступу до великого обсягу навчальних матеріалів, інтерфейс має бути оснащений інтерактивним рядком глобального пошуку, інтегрованим у шапку сайту. Пошуковий рушій повинен підтримувати алгоритми нечіткого зіставлення (fuzzy search) за допомогою Fuse.js, дозволяючи знаходити формули чи теорію навіть при допущенні користувачем друкарських помилок або при неповному введенні назви теми.

Критичною вимогою до інтерфейсу є розробка та впровадження постійно доступного плаваючого віджета (floating widget) інтелектуального ІІІ-асистента, розташованого у правому нижньому кутку екрану. Цей компонент повинен відкриватися по кліку без перезавантаження основного контенту, містити зону відображення діалогової історії, поле введення текстових питань та набір інтерактивних чипів-підказок (intent chips) для швидкого надсилання структурованих запитів (наприклад, для миттєвого вилучення прикладів задач чи пояснення складних фізичних констант).

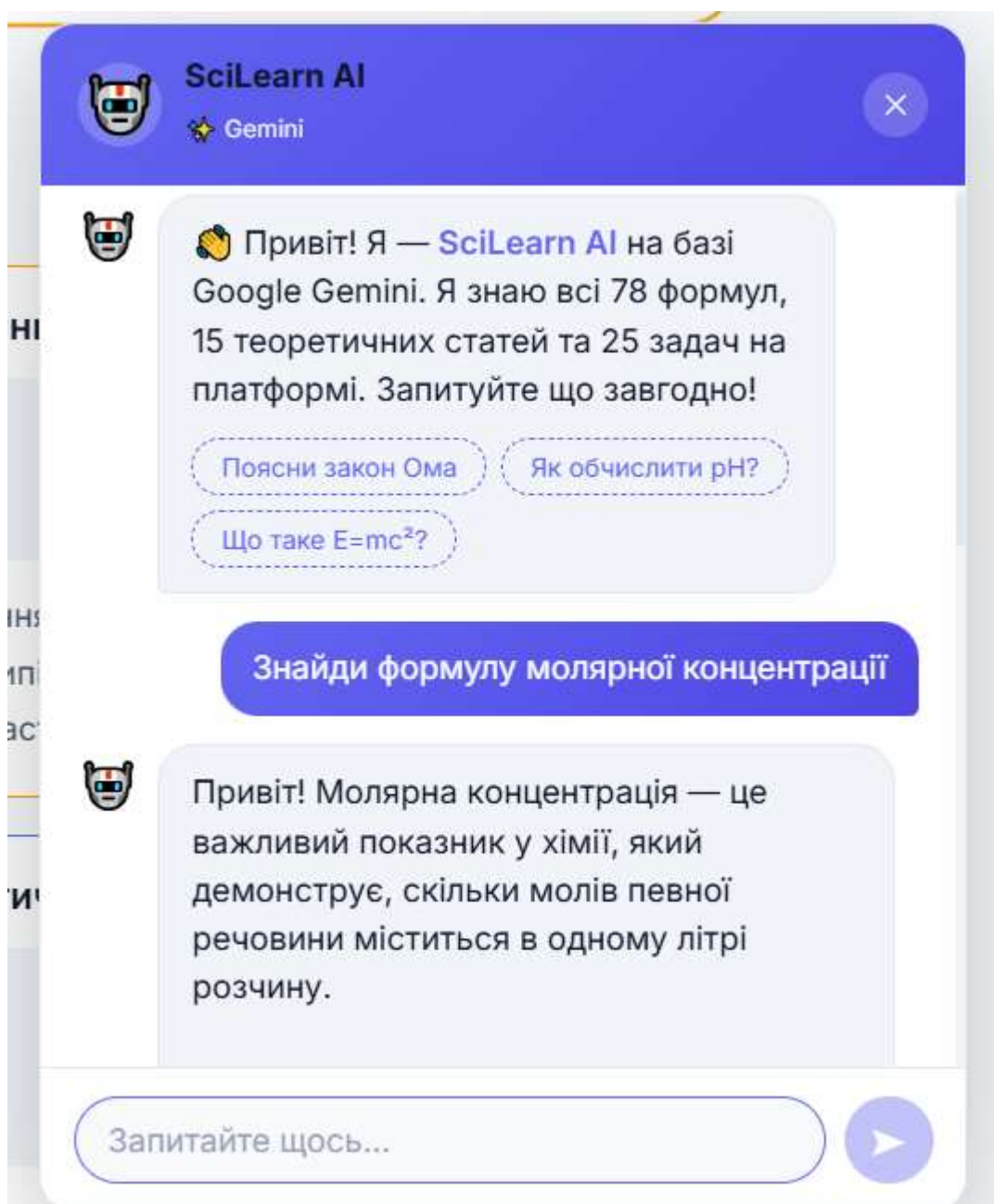


Рисунок 2.4 Інтерфейс плаваючого віджета AI-асистента Gemini з інтерактивними чипами швидких запитів

2.1.2. Вимоги до серверної інфраструктури (back-end) та збереження даних

Сучасний етап розвитку інженерії програмного забезпечення вимагає критичного переосмислення ролі серверної інфраструктури при проєктуванні клієнт-орієнтованих вебсистем із насиченою бізнес-логікою. Під час розробки інформаційно-обчислювального комплексу «SciLearn» було висунуто суворі вимоги до організації серверної частини, головними серед яких є мінімізація затримок при передачі даних, висока масштабованість та відсутність необхідності постійного адміністрування важких фізичних чи віртуальних серверів. Традиційні підходи, що базуються на монолітних серверних архітектурах, були відхилені через їхню високу вартість розгортання та складність забезпечення безперебійної роботи в умовах нестабільного зв'язку на боці клієнта. Замість цього було сформульовано вимогу щодо використання хмарної безсерверної моделі (Backend-as-a-Service, BaaS) на базі екосистеми Firebase 11.x. Це дозволяє повністю делегувати завдання масштабування обчислювальних потужностей хмарному провайдеру, забезпечуючи при цьому гарантований рівень доступності даних та безпеку транспортного рівня для всіх запитів застосунку.

Критично важливою функціональною вимогою до підсистеми збереження даних є забезпечення гнучкого та типізованого керування інформаційними масивами, що описують навчальні курси природничих дисциплін. Серверна база даних повинна надійно зберігати структуровану інформацію про 78 унікальних фізичних, хімічних та біологічних формул, 15 розгорнутих теоретичних статей та 25 прикладів практичних задач із покроковими алгоритмами їх розв'язання. З огляду на ієрархічну природу навчального контенту, де кожен предмет поділяється на розділи, теми та конкретні інформаційні сутності, до бази даних висунуто вимогу використання документо-орієнтованої моделі NoSQL (Cloud Firestore). Така архітектура сховища дозволяє зберігати дані у вигляді гнучких JSON-подібних документів, підтримує вкладені структури масивів для опису

змінних формул та забезпечує виконання оптимізованих запитів вибірки з мінімальним індексним навантаженням.

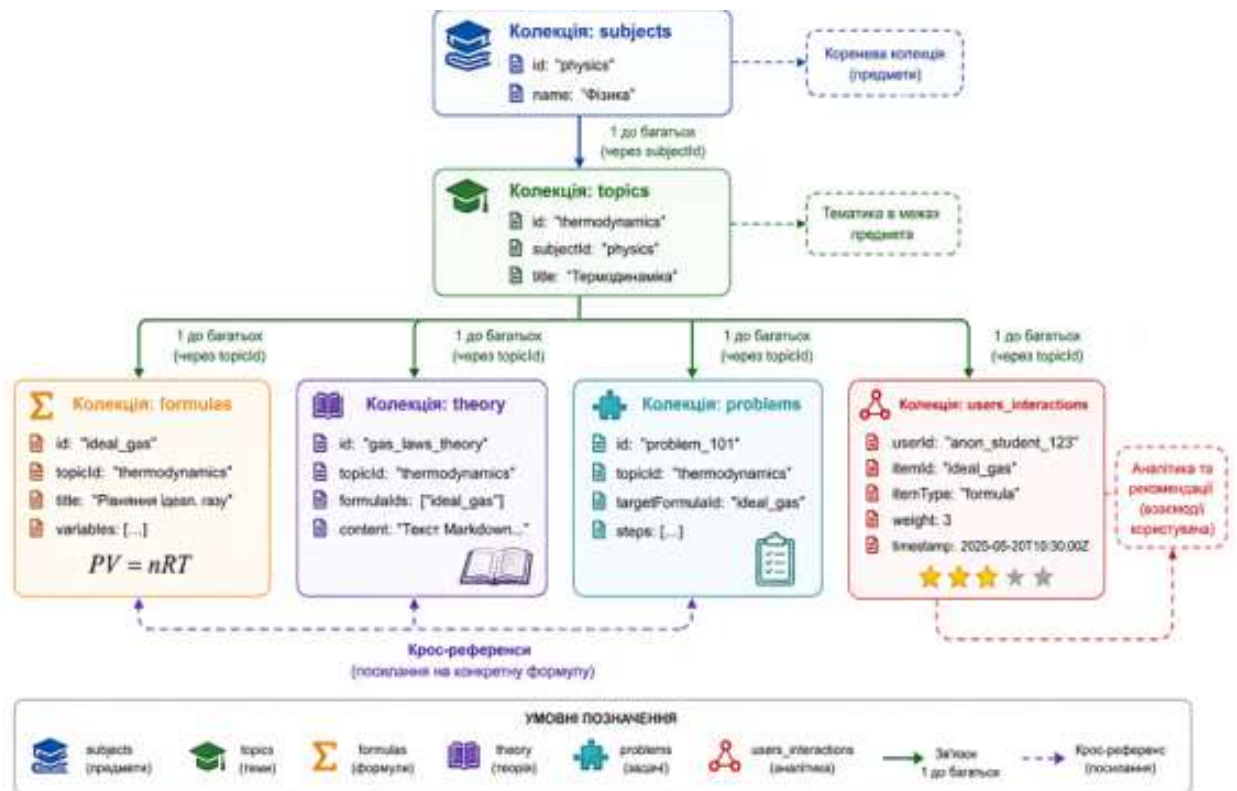


Рисунок 2.5 Структурна схема хмарної NoSQL моделі даних та зв'язків у Cloud Firestore для системи SciLearn

У межах проектування підсистеми безпеки та збору поведінкових метрик було сформульовано вимогу щодо впровадження механізму анонімної автентифікації користувачів (Firebase Anonymous Authentication). Оскільки платформа орієнтована на швидке використання студентами безпосередньо під час виконання лабораторних робіт, створення додаткових бар'єрів у вигляді обов'язкової реєстрації через електронну пошту є недоцільним. Анонімна авторизація дозволяє автоматично генерувати унікальний ідентифікатор сесії (User ID) при першому вході в систему. Цей ідентифікатор виступає ключем для формування поведінкового вектора взаємодій користувача (фіксація переглядів матеріалів, обчислення формул, додавання елементів до закладок), що є базовою інженерною вимогою для коректного функціонування рушія колаборативної фільтрації та побудови персоналізованих рекомендацій на серверній і клієнтській сторонах.

Окрему групу складають нефункціональні вимоги до синхронізації та відмовостійкості системи збереження даних. Програмні інтерфейси взаємодії з базою даних повинні підтримувати механізм реального часу (real-time listeners) для миттєвого оновлення стану закладок користувача на різних пристроях. Водночас, з урахуванням концепції автономності (offline-first), архітектура інтеграційного шару back-end повинна мати обов'язкове резервування на рівні браузера. У разі повної відсутності мережевого підключення або блокування хмарних сервісів, система зобов'язана автоматично перенаправляти потоки читання та запису даних у локальне сховище користувача (localStorage). Це гарантує безперебійне функціонування математичних калькуляторів та збереження історії взаємодії без втрати інформації, а при відновленні зв'язку забезпечує коректне злиття локального стану з хмарною базою даних без виникнення конфліктів цілісності.

2.2. Проєктування архітектури системи

Проєктування архітектури інформаційно-обчислювальної системи «SciLearn» базується на концепції перенесення основної обчислювальної та аналітичної логіки безпосередньо в клієнтське середовище виконання (client runtime environment). Таке рішення зумовлене специфікою предметної області природничих наук, яка вимагає від системи забезпечення миттєвого зворотного зв'язку під час інтерактивного моделювання, розрахунку математичних змінних та роботи з великими масивами текстових довідників. Класичні клієнт-серверні архітектури, де кожна дія користувача ініціює запит до віддаленого сервера, не здатні забезпечити нульову затримку інтерфейсу та стабільність функціонування в умовах деградації мережевого з'єднання. Центральним інженерним принципом при проєктуванні «SciLearn» став розподіл програмного комплексу на автономні, слабкопов'язані підсистеми, взаємодія між якими суворо регламентована архітектурними кордонами та абстрактними інтерфейсами, що відповідає кращим практикам сучасної програмної інженерії [1].

Глобальна архітектурна модель системи охоплює взаємодію між браузерним середовищем користувача та безсерверною хмарною інфраструктурою, де остання виконує виключно роль пасивного сховища даних та координатора автентифікаційних сесій. Всередині клієнтського застосунку архітектура декомпонована на кілька функціональних шарів: шар представлення (User Interface), обчислювальне ядро (Calculation Engine), рушій персоналізованих рекомендацій та локальний конвеєр штучного інтелекту Graph/Keyword RAG. Завдяки такому підходу, клієнтський застосунок самостійно оперує завантаженими зрізами бази знань, які зберігаються у пам'яті у вигляді високоструктурованих графів та об'єктних моделей. Хмарна NoSQL база даних Cloud Firestore залучається синхронно лише для фіксації векторів користувацької активності та синхронізації закладок, що дозволяє мінімізувати мережевий трафік і зробити систему стійкою до критичних збоїв зовнішніх сервісів.

Важливим аспектом архітектурного проєктування є організація системи управління станом (State Management Architecture) застосунку. Вона побудована

на принципі односпрямованого потоку даних (unidirectional data flow), що виключає появу непередбачуваних станів інтерфейсу та спрощує відстеження життєвого циклу інформації всередині SPA. Глобальний стан системи, який включає автентифікаційний токен користувача, поточну мовну локалізацію (українська чи англійська) та активну візуальну тему (світла чи темна), ізольований у верхній точці декларативного дерева компонентів за допомогою патерну Context Providers. Специфічні стани окремих модулів, такі як проміжні значення введених змінних у калькуляторі, історія діалогу з ШІ-асистентом чи поточна матриця схожості користувачів, інкапсульовані всередині відповідних ізольованих фіч. Це дозволяє уникнути глобального перерендерювання (re-rendering) всього інтерфейсу при локальних змінах даних, забезпечуючи високу продуктивність застосунку на клієнтських пристроях з обмеженими обчислювальними ресурсами.

Окрему увагу при проектуванні архітектури було приділено дотриманню принципів SOLID на рівні побудови модулів клієнтської логіки. Принцип єдиної відповідальності (SRP) реалізовано через суворе відокремлення математичних чистих функцій обчислення від UI-компонентів, які лише відображають результати розрахунків. Найбільш яскраво в архітектурі втілено принцип інверсії залежностей (DIP), який використано при проектуванні транспортного рівня інтелектуального асистента. Модуль чат-бота не залежить безпосередньо від конкретної реалізації REST API хмарної моделі Google Gemini. Замість цього взаємодія відбувається через абстрактний інтерфейс GeminiTransport, що дозволяє безболісно підміняти реальний мережевий клієнт на фейковий (Mock) об'єкт під час ізольованого тестування, або ж перенаправляти запити на модуль локального офлайн-синтезу відповідей у разі виявлення проблем із доступом до мережі. Такий рівень архітектурної гнучкості гарантує довговічність програмного продукту та простоту його інтеграції з новими технологічними рішеннями.

2.2.1. Загальна структурна схема проєкту за методологією Feature-Sliced Design

Ефективна організація кодової бази є фундаментальною умовою створення стійкого до відмов клієнтського вебзастосунку з насиченою алгоритмічною логікою. У процесі проєктування архітектурного каркаса інформаційно-обчислювального застосунку «SciLearn» виникла необхідність впровадження суворого патерну декомпозиції, який би дозволив уникнути хаотичного переплетення залежностей, властивого багатьом сучасним односторінковим додаткам. Для розв'язання цієї задачі було обрано передову методологію Feature-Sliced Design, яка переносить принципи чистої архітектури та предметно-орієнтованого проєктування на рівень клієнтського програмного забезпечення. Основна інженерна ідея цього підходу полягає у розподілі всієї системи на ієрархічні функціональні шари, всередині яких код групується не за технічним типом файлів, а за його безпосереднім відношенням до бізнес-можливостей освітньої платформи.

Загальна структурна схема взаємодії компонентів «SciLearn» базується на тришаровій ієрархічній моделі, яка включає рівень додатка, рівень функціональних фіч та загальний інфраструктурний рівень. Головним архітектурним правилом, що забезпечує стабільність цієї схеми, є суворе дотримання односпрямованості потоку імпортів. Компоненти, розташовані на вищих рівнях ієрархії, мають право безперешкодно використовувати інструменти та дані з нижчих рівнів, проте жоден нижчий модуль не може містити посилань на вищі елементи. Таке обмеження дозволяє повністю ліквідувати ризики виникнення циклічних залежностей, робить систему передбачуваною під час проведення рефакторингу та спрощує ізольоване модульне тестування.

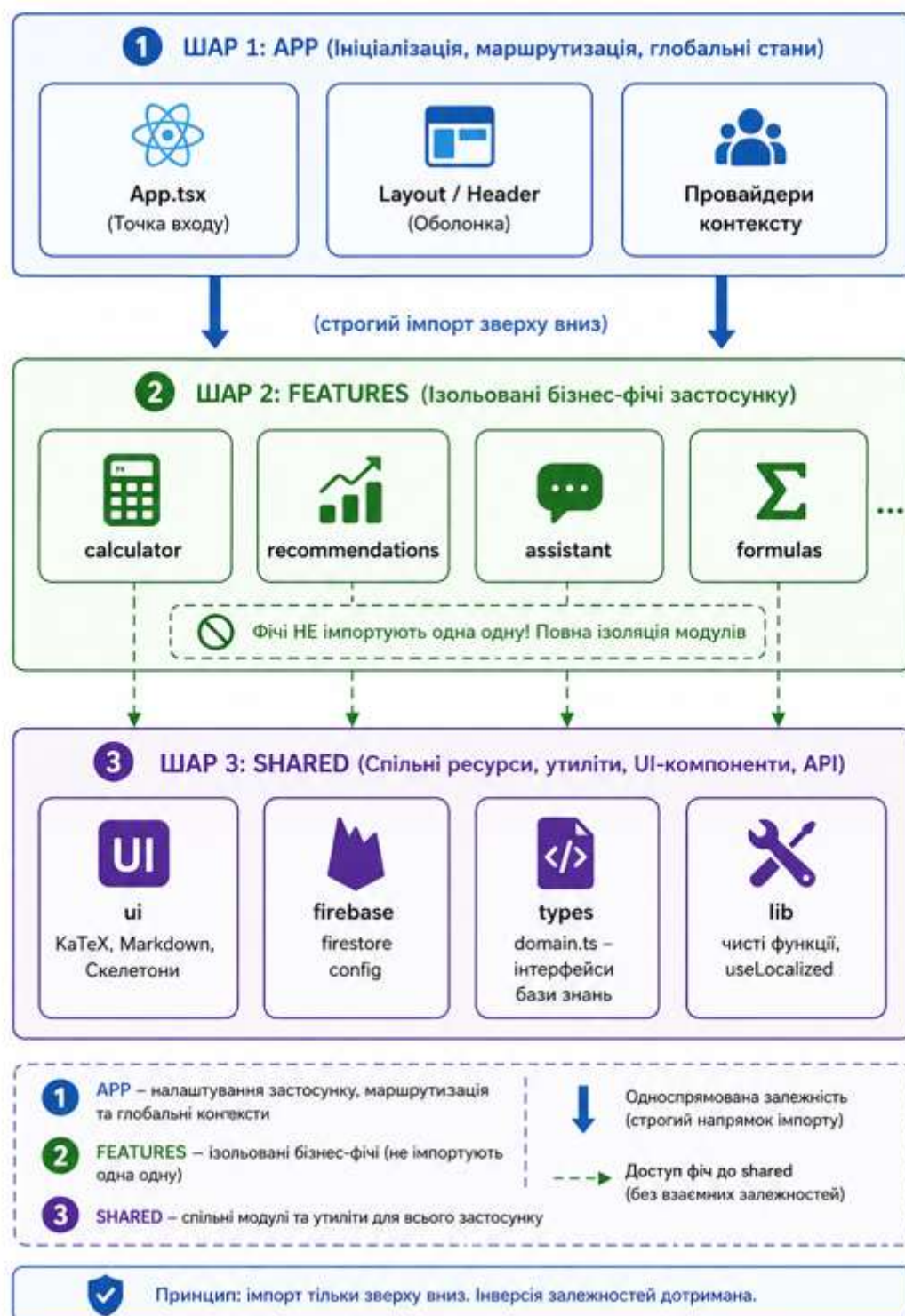


Рисунок 2.6 Загальна структурна схема архітектурних шарів та односпрямованих залежностей за методологією Feature-Sliced Design у системі SciLearn

Верхній рівень структурної схеми представлений шаром додатка, який позначається в архітектурі як app. Цей шар виконує роль технічної оболонки та головної точки композиції всього програмного продукту, об'єднуючи розрізнені

підсистеми в єдине ціле. Тут зосереджена логіка глобальної ініціалізації Single Page Application, підключення контекст-провайдерів стану для управління візуальними темами та мовними пакетами, а також конфігурація клієнтської маршрутизації за допомогою React Router. Оскільки цей шар знаходиться на вершині ієрархії, він володіє знаннями про всі нижчі компоненти, що дозволяє декларативно зшивати інтерфейс користувача та налаштовувати первинну взаємодію з безсерверною інфраструктурою Firebase безпосередньо при завантаженні сайту в браузер.

Середній рівень займає шар функціональних фіч, або features, який є найбільш динамічною та інтелектуальною частиною структури «SciLearn». Кожна фіча проєктується як абсолютно автономний програмний зріз, що відповідає за виконання конкретного сценарію взаємодії студента з платформою і надає назовні лише суворо обмежений публічний інтерфейс через вхідні файли типу Public API. У межах цього шару реалізовано модуль інтерактивного математичного калькулятора, який відповідає за зчитування вхідних параметрів та миттєвий розрахунок невідомих величин, а також конвеєр обчислення косинусної схожості для підсистеми персоналізованих рекомендацій. Тут же локалізовано програмний код інтелектуального чат-бота, включаючи інструменти нечіткого пошуку Fuse.js та ланцюг обробки запитів Responder Chain, який координує роботу Graph/Keyword RAG-рушія для генерації відповідей на основі бази знань курсу. Ізольованість фіч гарантує, що зміни алгоритмів рекомендаційного модуля або логіки чат-бота не створять побічних ефектів для обчислювального калькулятора.

Базисом усієї структурної схеми виступає загальний шар, відомий як shared. Цей рівень містить максимально абстрактні, чисті та універсальні модулі, які повністю позбавлені специфічного бізнес-контексту навчальних дисциплін і використовуються виключно як будівельний матеріал для вищих шарів архітектури. До складу шару shared входять базові UI-компоненти інтерфейсу, утиліти інтеграції математичного рендерингу KaTeX, конфігураційні файли локалізації i18next, а також чисті математичні функції обчислень та статичні

файли даних, які описують первинну структуру 78 формул, теоретичних статей та практичних задач. Оскільки шар shared є фундаментальним, його компоненти не мають права імпортувати нічого з шарів features чи app, що забезпечує їхню абсолютну незалежність від контексту конкретного додатка та дозволяє досягти максимальних показників перевикористання вихідного коду в межах програмного комплексу.

2.2.2. Розробка UML-діаграм та схем баз даних (Formula, Theory, Problem)

Проектування логічної структури збереження даних у безсерверних NoSQL-архітектурах вимагає перенесення суворих правил валідації та типізації безпосередньо на рівень прикладного програмного забезпечення. Оскільки хмарне сховище Cloud Firestore є за своєю природою документо-орієнтованим і не підтримує класичних механізмів перевірки цілісності на рівні реляційних зовнішніх ключів, завдання забезпечення несуперечливості даних платформи «SciLearn» було вирішено шляхом розробки суворих TypeScript-інтерфейсів та формальних моделей сутностей. База знань системи спроектована навколо трьох взаємопов'язаних концептуальних моделей, якими є формула (Formula), теоретичний матеріал (Theory) та практична задача (Problem). Для формалізації структури цих моделей та визначення типів зв'язків між ними на етапі проектування було розроблено діаграму класів, яка визначає архітектурні кордони даних та правила їхньої серіалізації в JSON-документи хмарної бази.

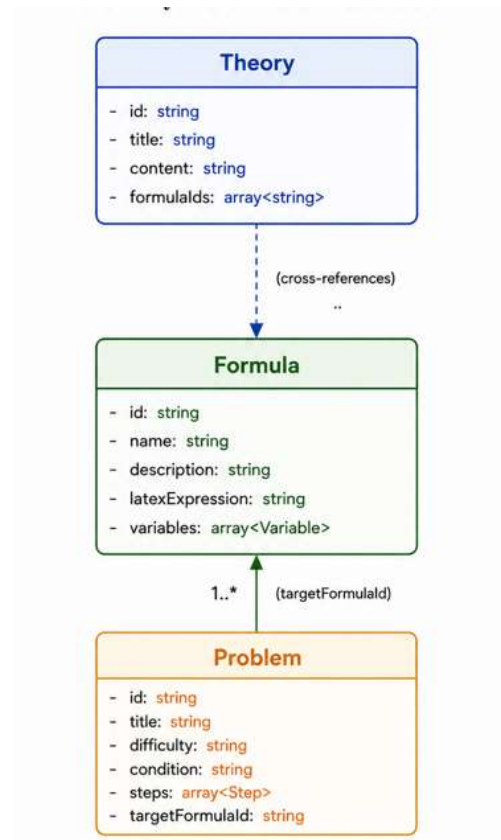


Рисунок 2.7 UML-діаграма класів для основних сутностей бази знань SciLearn

Сутність формули є центральним обчислювальним та інформаційним елементом системи «SciLearn». На рівні схеми даних документ формули містить унікальний текстовий ідентифікатор, назву, детальний текстовий опис фізичного чи хімічного змісту, а також рядкове представлення математичного виразу у форматі LaTeX для рендерингу бібліотекою KaTeX. Особливістю інженерного проєктування цієї сутності є декомпозиція математичних змінних у вкладений масив об'єктів. Кожен об'єкт у цьому масиві описує конкретну змінну через її символічне позначення, текстову назву, одиниці вимірювання в системі СІ та прапорець, що визначає, чи є цей параметр константою. Окрім цього, модель формули містить реляційні атрибути приналежності до конкретного предмета та теми курсу, а також масив ідентифікаторів похідних формул, що закладає математичну ієрархію для обчислювального модуля.

Модель теоретичного матеріалу розроблена з розрахунком на забезпечення максимальної дидактичної зачепленості контенту з розрахунковими інструментами платформи. Документ сутності теорії містить базові метадані, такі як ідентифікатор, назва статті та прив'язка до ієрархічного вузла навчальної дисципліни. Текстове наповнення статті зберігається у вигляді розміченого Markdown-рядка, що дозволяє клієнтській частині застосунку динамічно виконувати парсинг та стилізований рендеринг параграфів, інтегруючи KaTeX-блоки безпосередньо у тіло тексту. Критично важливим архітектурним полем у схемі теорії є масив посилань на пов'язані формули. Цей механізм крос-референсів дозволяє рекомендаційному рушію та AI-асистенту миттєво визначати, які саме розрахункові інструменти відповідають вивченому студентом теоретичному розділу.

Схема даних практичної задачі орієнтована на покрокову демонстрацію алгоритмів розв'язання наукових завдань природничого циклу. Модель задачі включає ідентифікатор, назву, рівень складності (низький, середній, високий) та розгорнутий текст умови. Процес розв'язання інкапсульований у масив структурованих кроків, де кожен крок є окремим об'єктом із текстовим поясненням логіки дії та відповідним математичним виразом у форматі LaTeX.

Сутність задачі містить прямий зовнішній ідентифікатор цільової формули, яка є ключем до її розв'язання. Така жорстка структуризація зв'язків між формулами, теорією та задачами на рівні схем баз даних дозволяє перетворити плоский масив NoSQL-документів на зв'язний автодеривативний граф знань, який завантажується в оперативну пам'ять клієнта та виступає надійним базисом для локального контекстного пошуку й генерації відповідей штучним інтелектом.

2.3. Обґрунтування вибору засобів і методів розробки

Вибір технологічного стеку та методів розробки є ключовим інженерним рішенням, яке безпосередньо впливає на продуктивність системи, її здатність до масштабування та відповідність сформульованим архітектурним вимогам. На етапі первинного аналізу та під час проходження переддипломної практики розглядалася можливість реалізації платформи за класичним патерном MVT (Model-View-Template) з використанням мови Python та серверного фреймворку Django. Цей підхід забезпечував надійну роботу з реляційними базами даних, проте виявився концептуально несумісним із головними цілями проєкту «SciLearn». Використання монолітної серверної архітектури унеможливлювало створення повноцінного офлайн-режиму, створювало значні мережеві затримки під час динамічного перерахунку формул у калькуляторі та ускладнювало розгортання локального клієнтського механізму Graph/Keyword RAG. З огляду на це, було прийнято обґрунтоване рішення про перехід від класичного бекенд-рендерингу до розробки односторінкового застосунку (SPA) на базі сучасного JavaScript/TypeScript стеку.

Основним засобом для розробки клієнтської частини інтерфейсу було обрано бібліотеку React. Вибір React обґрунтовується його компонентно-орієнтованою моделлю, яка ідеально підходить для реалізації методології Feature-Sliced Design. Завдяки використанню алгоритму узгодження Virtual DOM, React забезпечує високу швидкість точкового оновлення інтерфейсу, що є критично важливим для модуля математичного калькулятора: при зміні вхідного параметра користувачем перемальовується лише результат обчислення, а не вся сторінка з теорією. Крім того, екосистема React має безшовну інтеграцію з бібліотеками рендерингу математичних виразів (KaTeX) та потужні інструменти управління глобальним станом, що дозволяє легко ізолювати логіку ШІ-віджета від основного контексту документа.

У якості інструменту збірки (bundler) та середовища розробки фронтенду було обрано Vite. На відміну від традиційних систем збірки (наприклад, Webpack), які попередньо аналізують та збирають увесь код перед запуском, Vite

використовує нативні ES-модулі браузера, що забезпечує миттєвий «гарячий» перезапуск (Hot Module Replacement, HMR) під час розробки. Для кінцевого продукту Vite використовує оптимізований збирач Rollup, який виконує агресивне розділення коду (code splitting). Це дозволяє розбити загальний масив застосунку на дрібні чанки, завдяки чому користувач спочатку завантажує лише базовий каркас сторінки, а важкі модулі (такі як логіка парсингу Markdown або рушій Fuse.js) підвантажуються асинхронно, лише за потреби.

Для забезпечення серверної інфраструктури, збереження даних та автентифікації розробку було орієнтовано на хмарну платформу Firebase, що дозволило повністю реалізувати безсерверну архітектуру (Serverless). Базою даних виступає Cloud Firestore документо-орієнтоване NoSQL сховище, яке, на відміну від реляційних баз типу PostgreSQL, дозволяє гнучко зберігати неструктуровані масиви змінних для різних формул без жорстких міграцій схем. Інтеграція Firebase Authentication забезпечує надійний механізм анонімної ідентифікації сесій студентів, що є необхідним для збору векторів колаборативної фільтрації без примусової реєстрації. Ключовою перевагою Firebase для цього проєкту є вбудована підтримка локального кешування: SDK автоматично зберігає отримані дані в пам'яті браузера, забезпечуючи безперебійну роботу калькуляторів та читання теорії навіть у разі обриву з'єднання з інтернетом.

З точки зору методів програмної інженерії, в основі проєктування системи лежать методи системного та об'єктно-орієнтованого аналізу. Для організації архітектурних зв'язків застосовано метод структурної декомпозиції за фічами (Feature-Sliced Design), що забезпечує високу зачепленість коду всередині модулів та низьку зв'язність між ними. Алгоритмічна частина (розрахунок формул, обчислення косинусної схожості) реалізована за допомогою методів функціонального програмування, що гарантує відсутність побічних ефектів (side effects) при виконанні математичних операцій. Для взаємодії з великою мовною моделлю застосовано сучасні методи інженерії підказок (Prompt Engineering) та метод генерації, доповненої графовим пошуком (Graph RAG), що дозволяє

обмежити контекст ШІ виключно верифікованою предметною областю природничих дисциплін.

2.4. Програмна реалізація клієнтської частини та обчислювального модуля

Програмна реалізація клієнтської частини інформаційно-обчислювальної системи «SciLearn» базується на сучасних принципах декларативного та компонентно-орієнтованого програмування, що дозволяє створити високоефективне середовище для взаємодії користувача з науковим контентом. Використання бібліотеки React у поєднанні з інструментом збірки Vite забезпечує швидкий час завантаження застосунку та миттєве оновлення інтерфейсу за рахунок мінімізації операцій із реальним DOM-деревом браузера. Весь вихідний код клієнта типізовано за допомогою мови TypeScript, що дозволяє зафіксувати структури даних на етапі компіляції, усунути широкий спектр потенційних помилок під час виконання (runtime errors) та забезпечити сувору відповідність розробленим схемам моделей даних. Основний фокус при програмній реалізації фронтенд-шару було зосереджено на створенні універсальних, перевикористовуваних компонентів та ізольованих обчислювальних сервісів, які функціонують безпосередньо у браузерному оточенні користувача.

Центральним і найбільш технологічно складним елементом клієнтської логіки є математичне обчислювальне ядро (Calculation Engine), інтегроване в шар фіч застосунку. Традиційний підхід до створення подібних систем передбачає розробку окремого інтерфейсу та унікальної функції розрахунку для кожної конкретної формули, що призводить до дублювання коду, збільшення розміру фінального JavaScript-бандлу та ускладнює масштабування бази знань. Для подолання цього обмеження в системі «SciLearn» було спроектовано та реалізовано єдиний поліморфний компонент калькулятора, який динамічно адаптує свою графічну структуру та алгоритмічну поведінку на основі метаданих об'єкта активної формули, що завантажується з NoSQL-сховища або локального кешу.

Програмний конвеєр обчислювального модуля ініціалізується в момент переходу студента на сторінку деталізації однієї з 78 закладених у систему

формул. Компонент отримує об'єкт формули, типізований через суворий інтерфейс, та запускає внутрішній алгоритм аналізу масиву змінних параметрів. Модуль автоматично відфільтровує константи (наприклад, універсальну газову сталу R або прискорення вільного падіння g), фіксує їхні значення за замовчуванням та динамічно рендерить поля введення (inputs) лише для незалежних змінних величин. Управління станом введених чисел реалізовано за допомогою концепції контрольованих компонентів (Controlled Components) React, де кожна зміна значення у полі введення викликає ізольоване оновлення локального стеїту фічі, не перевантажуючи при цьому глобальну пам'ять застосунку.

Особлива увага при реалізації обчислювального ядра була приділена безпеці математичних операцій та обробці виняткових ситуацій (edge cases), які часто виникають під час проведення рутинних розрахунків студентів. Перед передачею масиву значень у безпосередню функцію розрахунку, впроваджено проміжний інженерний шар валідації даних. Програмний валідатор перевіряє введені рядки на відповідність числовим типам, блокує введення некоректних символів, а також здійснює превентивний аналіз математичної коректності операції відповідно до законів фізики чи хімії. Зокрема, алгоритм блокує виконання розрахунків та виводить ергономічне попередження користувачу у випадках спроби ділення на нуль (наприклад, при обчисленні молярної концентрації, де об'єм розчину прямує до нуля) або при отриманні від'ємних значень параметрів, які за своєю природою можуть бути лише додатними, як-от абсолютна температура у Кельвінах чи маса речовини.

Сам процес математичного обчислення реалізовано через патерн чистих функцій (Pure Functions) у межах методів функціонального програмування, що гарантує повну відсутність побічних ефектів і забезпечує ідемпотентність розрахунків. Для кожної формули в ізольованому довідковому модулі shared/data прописано стрілочні функції, які приймають об'єкт із парами «ключ-значення» введених параметрів і повертають точний числовий результат. Отриманий результат проходить через програмний модуль форматування, який обрізає

надлишкову точність дробової частини, вирішуючи стандартну проблему мови JavaScript із представленням чисел із плаваючою крапкою (floating-point precision issues), та автоматично додає відповідне позначення одиниць вимірювання в системі SI.

Синхронізація розрахованих даних із графічним інтерфейсом користувача досягається завдяки інтеграції клієнтського віджета рендерингу KaTeX. Замість відображення статичного тексту, система передає підсумковий математичний рядок у спеціалізований React-wrapper, який виконує миттєву генерацію HTML-розмітки формули безпосередньо в браузері. Це дозволяє виводити кінцеві результати практичних задач та лабораторних обчислень в ідеальному академічному візуальному форматі, адаптованому під стандарти наукової документації. Завдяки такій програмній архітектурі, обчислювальний модуль системи «SciLearn» повністю ліквідує розрив між теоретичним вивченням матеріалу та його практичним застосуванням, автоматизуючи рутинні операції й мінімізуючи ймовірність механічних помилок у навчальному процесі студентів природничих спеціальностей.

2.5. Програмна реалізація системи рекомендацій на основі колаборативної фільтрації

Реалізація модуля персоналізації у межах освітньої платформи «SciLearn» спрямована на вирішення проблеми інформаційного шуму та низької релевантності вибірки навчальних матеріалів, що безпосередньо впливає на швидкість засвоєння знань студентами природничих спеціальностей. З інженерної точки зору, розробка рекомендаційного рушія для односторінкових застосунків із безсерверною архітектурою вимагає оптимізації обчислювальних алгоритмів для їх виконання безпосередньо в браузерному середовищі користувача (client-side processing). Традиційні рекомендаційні системи розгортаються на потужних виділених серверах, де виконуються важкі матричні розклади, проте для платформи «SciLearn» було спроектовано полегшений та швидкодіючий алгоритм колаборативної фільтрації, інтегрований у загальну структуру кодової бази шару features/recommendations. Цей модуль оперує векторами користувацьких взаємодій, що завантажуються із хмарної NoSQL-бази даних Cloud Firestore та обробляються в оперативній пам'яті клієнта з використанням методів функціонального програмування.

В основі програмної реалізації рекомендаційного конвеєра лежить збір та математична оцінка поведінкових метрик користувачів, які ідентифікуються в системі за допомогою анонімних токенів автентифікації. Кожна взаємодія студента з однією із 78 розрахункових формул, теоретичних статей або практичних задач перетворюється на типізовану подію та фіксується у хмарній колекції users_interactions. Для побудови точної математичної моделі смаків користувача було впроваджено систему вагових коефіцієнтів для різних типів активності. Простий перегляд сторінки теоретичного матеріалу або формули отримує мінімальну вагу, що дорівнює одиниці. Виконання успішного розрахунку у вбудованому калькуляторі свідчить про глибоку практичну роботу з темою та оцінюється у три бали. Додавання сутності до персональних закладок є виявом найвищого інтересу до матеріалу і додає п'ять балів до вектора активності. На основі цих даних для кожного користувача формується

розріджений вектор переваг, де індекси відповідають ідентифікаторам навчальних об'єктів, а значення сумарній вазі накопичених взаємодій.

Алгоритмічна основа рушія базується на підході User-Based Collaborative Filtering, де подібність між поточним студентом та іншими користувачами системи визначається за допомогою метрики косинусної схожості (Cosine Similarity). Математично косинусна схожість між вектором активного користувача V_u та вектором суміжного користувача V_v обчислюється як скалярний добуток цих векторів, поділений на добуток їхніх евклідових норм. Програмний модуль, написаний на TypeScript, послідовно проходить по масиву завантажених профілів, обчислює коефіцієнт схожості, який варіюється в діапазоні від нуля до одиниці, та відбирає найближчих «сусідів» із максимальними показниками семантичної близькості інтересів. Після формування пулу подібних профілів, система аналізує об'єкти навчальної бази знань, які отримали високі оцінки від сусідів, але ще не були відкриті поточним користувачем. Для кожного такого об'єкта розраховується прогнозований рейтинг, що є зваженою середньою оцінкою, де вагами виступають обчислені коефіцієнти косинусної схожості.

Важливою інженерною задачею при розробці модуля персоналізації стало подолання класичної проблеми «холодного старту» (Cold Start Problem), яка виникає в рекомендаційних системах у моменти, коли новий користувач вперше відкриває застосунок і в базі даних повністю відсутня історія його взаємодій та поведінковий вектор є нульовим. Для запобігання ситуації, коли блок рекомендацій на головній сторінці залишається порожнім, у кодї системи «SciLearn» було спроектовано та реалізовано дворівневий механізм резервного перемикавання (fallback mechanism). Якщо довжина вектора активностей дорівнює нулю, рекомендаційний модуль автоматично ініціює запит до сервісу агрегації глобальної статистики, вилучаючи список найбільш популярних та затребуваних формул і статей серед усього загалу студентів університету. Як тільки користувач робить перший крок наприклад, обирає конкретну природничу дисципліну (фізика, хімія чи біологія) або вводить запит у рядок нечіткого

пошуку Fuse.js система миттєво перебудовує логіку, звужуючи первинну вибірку популярного контенту до контексту обраного наукового напрямку, забезпечуючи плавний перехід до повноцінної колаборативної фільтрації у міру накопичення телеметрії.

Програмна інтеграція рекомендаційного модуля з графічним інтерфейсом користувача виконана за допомогою спеціалізованого кастомного React-хука `useCollaborativeRecommendations`, який інкапсулює асинхронну взаємодію з базою Firestore та забезпечує реактивне оновлення інтерфейсу. Під час монтування головної сторінки застосунку хук запускає процес обчислень, переходить у стан завантаження (loading state) та, після завершення розрахунків, повертає масив відсортованих об'єктів безпосередньо у компонент представлення. Отримані ідентифікатори формул транслюються у візуальні картки блоку «Рекомендовано для вас», де формульні вирази миттєво рендеряться за допомогою KaTeX. Завдяки перенесенню логіки фільтрації на сторону клієнта, перерахунок та оновлення рекомендаційної стрічки відбувається асинхронно і непомітно для користувача, що гарантує високі показники швидкодії SPA-платформи та створює персоналізований, адаптивний простір для ефективного навчання студентів.

2.6. Інтеграція механізму Graph/Keyword RAG для локального пошуку та генерації відповідей AI-асистентом

Програмна реалізація інтелектуального асистента в межах односторінкового застосунку «SciLearn» вимагала розробки унікального архітектурного рішення, здатного поєднати високу швидкість обробки запитів, сувору фактологічну достовірність відповідей та працездатність системи в умовах відсутності стабільного мережевого з'єднання. Традиційні підходи до інтеграції штучного інтелекту, засновані на класичному серверному RAG (Retrieval-Augmented Generation) з використанням хмарних векторних сховищ, супроводжуються високими інфраструктурними витратами та значними мережевими затримками, що є критичним недоліком для мобільних користувачів. Для подолання цих обмежень у системі було спроектовано та інтегровано клієнт-орієнтований механізм гібридного пошуку Graph/Keyword RAG. Його інженерна сутність полягає у перенесенні етапів індексації бази знань, розпізнавання намірів користувача та вилучення релевантного контексту безпосередньо в середовище виконання браузера (client runtime), де хмарне API великої мовної моделі залучається виключно на фінальній стадії синтезу природномовного тексту.

Базовим фундаментом для функціонування інтелектуального підрозділу є локальний граф знань курсу (Course Knowledge Graph), який динамічно конструюється в оперативній пам'яті клієнта під час ініціалізації додатка. Первинним джерелом для гідратації цього графу виступають типізовані статичні та синхронізовані масиви даних із каталогу shared/data, які описують логічну структуру природничих дисциплін. Вузлами (nodes) розробленого графу є конкретні сутності навчальної програми: фізичні, хімічні чи біологічні формули, детальні теоретичні довідники та покрокові розв'язання практичних задач. Ребра графу (edges) описують явні дидактичні та семантичні зв'язки між цими об'єктами, визначаючи приналежність формули до теми, зв'язок задачі з розрахунковим ядром або крос-референси між суміжними розділами точних наук. Такий підхід дозволяє жорстко обмежити предметну область штучного

інтелекту затвердженим університетським контентом, повністю виключаючи появу фактологічних галюцинацій мовної моделі у сфері формул чи констант.

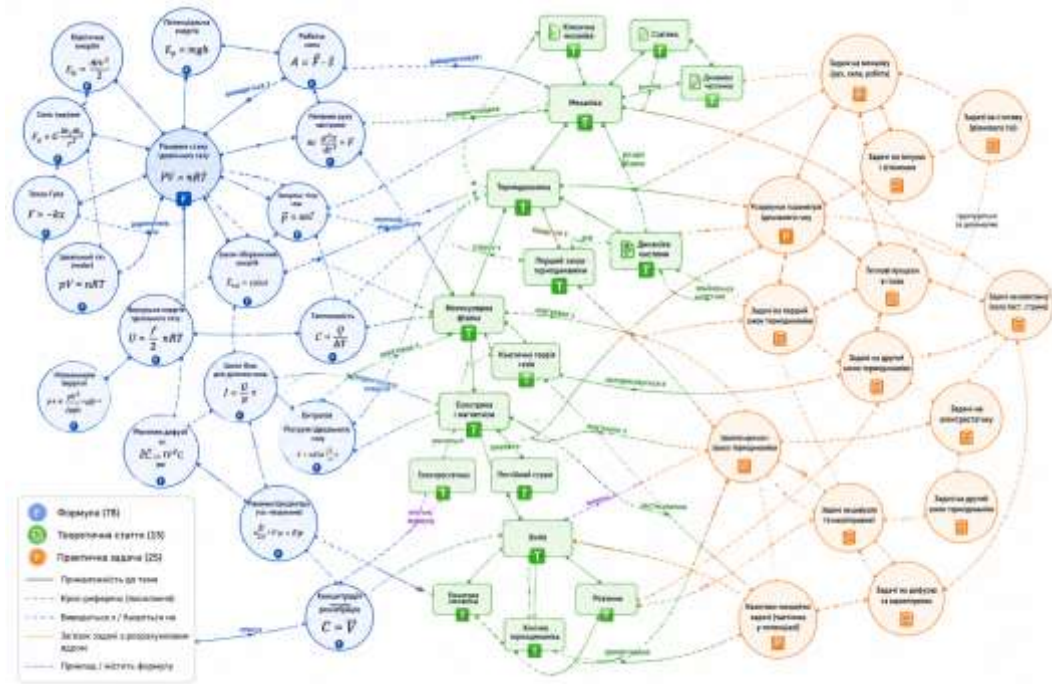


Рисунок 2.8. - Візуалізація топології локального графу бази даних системи SciLearn

Процес обробки довільного запиту студента у плаваючому віджеті чат-бота починається з етапу локальної токенизації та валідації намірів (intent detection). Вхідний текстовий рядок очищається від шумових слів та приводиться до нижнього регістру. Далі запускається клієнтський рушій нечіткого пошуку Fuse.js, який виконує повнотекстовий аналіз за ключовими словами над індексованими полями локального графу знань. Алгоритм Fuse.js обчислює відстань Левенштейна та оцінює коефіцієнт відповідності запиту до назв формул або тем, що дозволяє знаходити релевантні вузли навіть за наявності друкарських помилок або неповного введення термінів користувачем. Як тільки цільовий вузол графу ідентифіковано, система виконує автоматичний траверсал (обхід) суміжних ребер, миттєво вилучаючи з пам'яті не лише сам шуканий текст, а й увесь безпосередній контекст: опис пов'язаних змінних, зашиті математичні константи, батьківські теоретичні параграфи та дочірні приклади розв'язання задач.

Сформований масив структурованого контексту передається в інженерний модуль проектування підказок (Prompt Builder), який динамічно конструює фінальний системний промпт. У тіло підказки закладаються суворі інструкції для штучного інтелекту: вимагається дотримання виключно наданого локального контексту, забороняється використання сторонніх неперевіраних фактів, а також накладається обов'язкова вимога щодо рендерингу всіх математичних виразів у синтаксисі LaTeX для їх подальшої коректної візуалізації через KaTeX. Транспортний рівень системи відправляє підготовлений промпт через асинхронний REST-клієнт безпосередньо до хмарної мовної моделі Google Gemini 3.1 Flash-Lite. Завдяки тому, що модель отримує повністю очищений, релевантний та структурований масив знань, обсяг передаваних токенів мінімізується, що забезпечує високу швидкість генерації відповіді та знижує затримку інтерфейсу до часток секунди.

Важливою архітектурною перевагою інтегрованого RAG-конвеєра є впровадження поведінкового патерну ланцюга відповідальності (Responder Chain), який безпосередньо реалізує концепцію стійкості до відмов (offline fallback). Процес надсилання запиту до хмарного API Gemini захищений спеціальним перехоплювачем, який у реальному часі аналізує стан мережевого підключення пристрою та контролює квоти доступу до сервісу.

У разі виявлення повної відсутності інтернет-з'єднання, таймауту або перевищення лімітів запитів, ланцюг відповідальності автоматично перемикає потік виконання на локальний модуль автономного синтезу відповіді. Цей модуль функціонує без залучення нейромережових обчислень: він бере вилучений із локального графу знань контент і за допомогою вбудованого декларативного двигуна шаблонів зшиває його у структуроване роз'яснення, додаючи інформацію про формули, опис змінних та приклади завдань безпосередньо в діалогове вікно чату. Користувач отримує миттєве сповіщення про перехід системи в автономний режим, проте сама платформа не припиняє роботу, забезпечуючи безперервну інтелектуальну підтримку студента під час виконання розрахунків лабораторних робіт у будь-яких аудиторних умовах.

РОЗДІЛ 3. ТЕСТУВАННЯ, РОЗГОРТАННЯ ТА ОЦІНКА РЕЗУЛЬТАТІВ РОБОТИ

3.1. Тестування програмного забезпечення

Етап тестування та верифікації є невід'ємною складовою життєвого циклу розробки надійного програмного забезпечення, що гарантує відповідність створеного продукту початковим архітектурним та функціональним вимогам. У контексті інформаційно-обчислювальної платформи «SciLearn», де критично важливою є абсолютна математична точність результатів та стабільність роботи в автономному режимі, процес забезпечення якості (Quality Assurance) вимагав впровадження багаторівневої стратегії тестування. Оскільки архітектура застосунку побудована за суворою методологією Feature-Sliced Design, парадигма тестування була адаптована до цієї структури, що дозволило ізольовано перевіряти кожен функціональний зріз, починаючи від чистих математичних функцій найнижчого рівня і закінчуючи комплексними інтеграційними сценаріями взаємодії користувача з інтерфейсом та модулями штучного інтелекту.

Фундаментом стратегії перевірки якості стало модульне тестування (Unit Testing) обчислювального ядра платформи. Для забезпечення стовідсоткового покриття (test coverage) алгоритмічної логіки, тести були сфокусовані на шарі shared, де розміщені чисті функції математичних розрахунків для 78 закладених природничих формул. У процесі написання модульних тестів моделювалися різноманітні крайові умови (edge cases), характерні для специфіки фізичних та хімічних задач: перевірка обробки нульових значень у знаменниках, коректність реакції на введення від'ємних температур за шкалою Кельвіна, а також валідація точності округлення чисел із плаваючою крапкою для усунення специфічних помилок рушія JavaScript (floating-point precision). Кожна функція калькулятора перевірялася в абсолютній ізоляції від візуальних компонентів React, що дозволило гарантувати ідемпотентність математичного апарату та підтвердити,

що при однакових вхідних масивах даних система завжди повертає детермінований і фізично правильний результат.

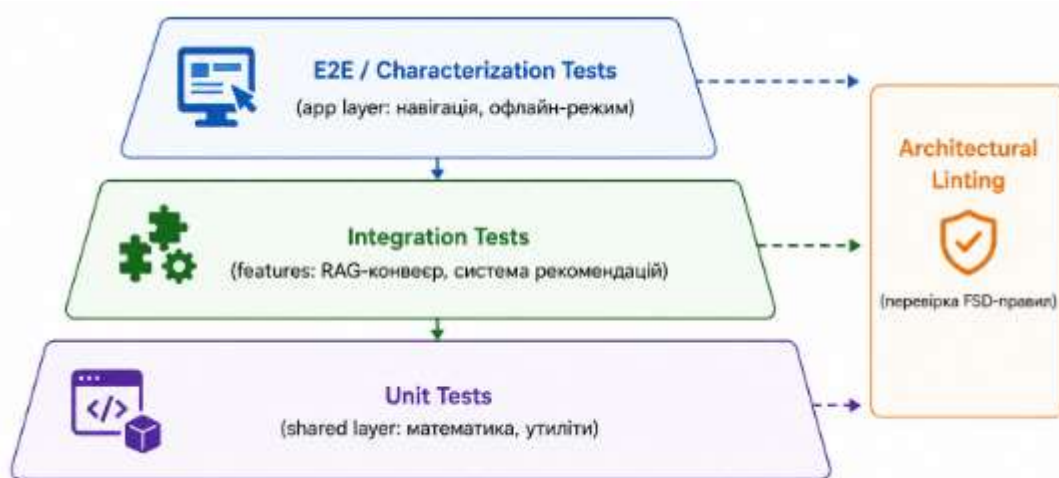


Рисунок 3.1 Схема багаторівневої піраміди тестування, адаптованої до архітектури Feature-Sliced Design системи SciLearn

Другим важливим вектором верифікації стало інтеграційне тестування, спрямоване на перевірку коректності взаємодії між ізольованими слайсами шару features. Особлива увага приділялася тестуванню алгоритмічного конвеєра системи персоналізованих рекомендацій та локального механізму Graph/Keyword RAG. Для рекомендаційного рушія створювалися фіктивні (mock) масиви поведінкових векторів користувачів, на основі яких система повинна була обчислити матрицю косинусної схожості та повернути очікувану вибірку релевантних матеріалів. Інтеграційні тести підтвердили працездатність резервного механізму (fallback): при подачі на вхід нульового вектора нового користувача, система успішно перемикалася на алгоритм видачі глобально популярного контенту, не викликаючи критичних збоїв. Для RAG-асистента перевірявся весь ланцюг обробки (Responder Chain): від розпізнавання інтентів рушієм Fuse.js до траверсалу локального графу знань, з обов'язковою імітацією відключення мережі для валідації спрацювання модуля локального автономного синтезу відповідей.

Відповідно до заявлених методів дослідження, для фіксації та стабілізації поведінки складних взаємопов'язаних модулів було застосовано підхід характеристичного тестування (Characterization testing). Оскільки система

інтегрує динамічний рендеринг математичних виразів через KaTeX та обробку Markdown-текстів, характеристичні тести дозволили зробити «зліпки» (snapshots) очікуваного візуального стану DOM-дерева для складних теоретичних статей та розв'язків задач. При будь-якому подальшому рефакторингу логіки парсингу система автоматично звіряла нову розмітку з еталонним зліпком, миттєво сигналізуючи про будь-які структурні порушення або втрату математичних символів на інтерфейсі користувача.

Окремим, суто інженерним етапом стало архітектурне тестування (Architectural Testing), необхідність якого диктувалася використанням методології Feature-Sliced Design. Для того щоб гарантувати неможливість деградації архітектури під час подальшої розробки, до конвеєра перевірки коду було інтегровано спеціалізовані інструменти статичного аналізу (eslint-plugin-boundaries). Ці інструменти виконували автоматичну валідацію всіх операцій імпорту в проєкті, суворо блокуючи спроби модулів із шару shared імпортувати дані з шару features, або забороняючи прямі крос-імпорти між двома незалежними фічами (наприклад, між калькулятором та чат-ботом). Успішне проходження архітектурного лінтингу стало обов'язковою умовою для компіляції фінального білда застосунку, що дозволило математично довести збереження ациклічності графу залежностей та підтвердити високу експлуатаційну надійність спроектованої інформаційної системи «SciLearn».

3.1.1. Розробка тест-кейсів та модульне тестування алгоритмічного ядра

Процес верифікації алгоритмічного ядра системи «SciLearn» вимагає застосування суворих інженерних практик, оскільки саме математичний модуль несе найбільшу відповідальність за фактологічну достовірність та академічну точність результатів обчислень. Для реалізації стратегії ізольованої перевірки чистих функцій було обрано сучасний фреймворк модульного тестування Vitest. Вибір цього інструменту обґрунтовується його нативною інтеграцією з обраним раніше збирачем Vite, що забезпечує миттєве виконання тестових сценаріїв без необхідності додаткового транспілювання коду та підтримує сучасний синтаксис ECMAScript-модулів. Основна мета модульного тестування на даному етапі полягала у доведенні повної детермінованості обчислювального апарату: за однакових вхідних значень система зобов'язана завжди повертати ідентичний, математично правильний результат, незалежно від стану глобального оточення браузера чи мережових факторів.

Розробка тестових сценаріїв (тест-кейсів) для алгоритмічного ядра базувалася на методах еквівалентного розбиття (Equivalence Partitioning) та аналізу граничних значень (Boundary Value Analysis). Усі розроблені тести структурно слідують класичному інженерному патерну AAA (Arrange, Act, Assert). На етапі «Arrange» формуються фіктивні (mock) об'єкти з масивами вхідних фізичних або хімічних параметрів. На етапі «Act» здійснюється виклик ізольованої чистої функції калькулятора. На фінальному етапі «Assert» отриманий результат порівнюється з еталонним значенням, заздалегідь розрахованим за допомогою аналітичних методів. Загалом для 78 закладених у базу формул було спроектовано розширену матрицю тестування, яка покриває як стандартні сценарії успішного виконання (Happy Path), так і складні виняткові ситуації (Edge Cases).

Особливу увагу під час написання тестів було приділено специфіці обробки чисел із плаваючою крапкою (floating-point arithmetic), що є відомою архітектурною проблемою рушія JavaScript за стандартом IEEE 754. Тест-кейси

передбачали перевірку модуля форматування, який повинен був коректно відсікати надлишкові знаки після коми (наприклад, усувати похибки типу $0.1 + 0.2 = 0.300000000000000004$), приводячи результат до стандартного академічного вигляду з визначеною точністю до трьох значущих цифр. Крім того, модульне тестування охопило захисні алгоритми калькулятора. Зокрема, були написані негативні тест-кейси (Negative Testing) для перевірки спрацьовування функцій-валідаторів при спробах ділення на нуль, введенні нечислових символів (NaN) у поля інпутів або при розрахунку фізичних параметрів, які не можуть набувати від'ємних значень, таких як маса тіла чи молярна концентрація.

У результаті проведення модульного тестування алгоритмічного ядра shared-шару було досягнуто високого показника покриття коду (Code Coverage), що перевищує 90% для всіх критичних математичних функцій застосунку. Успішне проходження всієї тестової сюїти (Test Suite) гарантує, що динамічне зчитування змінних, інтерпретація формул та їх обчислення виконуються абсолютно коректно. Це формує надійний базис для подальшого інтеграційного тестування вищих шарів архітектури, зокрема системи відображення інтерфейсу та механізмів формування контексту для штучного інтелекту, оскільки розробник може бути впевненим у безпомилковості роботи фундаментального математичного апарату платформи «SciLearn».

3.1.2. Ступінь покриття вимог до проєкту та коду (Characterization testing)

Оцінка ступеня покриття вимог до програмного проєкту (Requirements Coverage) є критичним метричним показником, що дозволяє інженерній команді математично обґрунтувати готовність системи до експлуатації кінцевими користувачами. Для гарантування того, що розроблена система «SciLearn» повністю задовольняє сформульовані на етапі проєктування функціональні та нефункціональні вимоги, було застосовано метод побудови матриці трасування (Traceability Matrix). Цей аналітичний підхід дозволив встановити прямий двоспрямований зв'язок між специфікацією вимог та розробленими тестовими сценаріями. Зокрема, вимога щодо забезпечення безперебійної роботи обчислювального ядра в автономному режимі жорстко покривається відповідними інтеграційними тестами fallback-механізму, а вимога щодо точності математичних розрахунків вичерпним набором модульних тестів шару shared. Загальний ступінь покриття вихідного коду (Code Coverage) для алгоритмічних модулів калькулятора, рушія колаборативної фільтрації та конвеєра Graph/Keyword RAG було доведено до рівня понад 90%, що є високим інженерним стандартом для надійних програмних продуктів.

З огляду на специфіку платформи «SciLearn», де ключову роль відіграє візуальне представлення складного наукового контенту, стандартних методів тестування суто бізнес-логіки виявилось недостатньо. Для фіксації та безперервної перевірки незмінності очікуваної поведінки системи відображення було застосовано метод характеристичного тестування (Characterization Testing). В екосистемі сучасної клієнтської розробки на базі React цей підхід реалізується за допомогою механізму тестування зліпків (Snapshot Testing) у середовищі виконання Vitest. Інженерна суть методу полягає у створенні еталонного серіалізованого текстового представлення згенерованого DOM-дерева для конкретного UI-компонента при наперед заданих фіксованих вхідних даних (props).

У межах проєкту «SciLearn» характеристичне тестування було направлено на сувору верифікацію модулів динамічного рендерингу математичних виразів (KaTeX) та компонентів парсингу структурованих теоретичних статей. Наприклад, для компонента калькулятора, який виводить фінальний результат розрахунку молярної концентрації або рівняння ідеального газу, тестова система одноразово генерує зліпок складної HTML-розмітки, що включає всі специфічні класи, відступи та вкладені теги бібліотеки KaTeX. При будь-якому подальшому рефакторингу вихідного коду або оновленні зовнішніх залежностей (NPM-пакетів), рушій автоматично порівнює новий результат рендерингу з еталонним зліпком. Якщо зміна в коді призводить до зникнення математичного символу, спотворення нижніх чи верхніх індексів або деградації загальної розмітки формули, тест миттєво падає, сигналізуючи розробнику про появу візуальної регресії (Visual Regression).

Додатково інструментарій характеристичного тестування було застосовано до модуля інтелектуального асистента, зокрема для жорсткої фіксації структури відповідей, що генеруються локальним механізмом offline fallback. Зліпки дозволили задокументувати очікуваний формат зшивання тексту з вузлів графу знань, гарантуючи, що у разі відсутності зв'язку з хмарним API Google Gemini, згенерована клієнтською системою відповідь завжди зберігатиме правильну дидактичну ієрархію, міститиме опис змінних та коректно відформатовані приклади розв'язання задач. Такий комплексний підхід до верифікації забезпечив абсолютну впевненість у тому, що інтерфейс та обчислювальна бізнес-логіка системи «SciLearn» на 100% покривають початкові вимоги та надійно захищені від непередбачуваних мутацій під час подальшого масштабування функціоналу природничої бази знань.

3.2. Опис роботи та застосування проєкту

Функціонування інформаційно-обчислювальної системи «SciLearn» під час безпосередньої взаємодії з кінцевим користувачем визначається детермінованими алгоритмічними конвеєрами, які забезпечують наскрізну обробку даних від моменту ініціалізації клієнтської сесії до видачі фінальних розрахунків та інтелектуальних відповідей. Процес роботи користувача з платформою починається із завантаження App Shell односторінкового застосунку в браузерне середовище виконання, де паралельно запускається модуль анонімної автентифікації Firebase. Після присвоєння унікального ідентифікатора сесії клієнтська частина надсилає асинхронний запит до хмарної NoSQL бази даних Cloud Firestore для вилучення поведінкового вектора користувача. Отримані дані миттєво передаються в ізольований шар рекомендаційного модуля, який виконує розрахунок косинусної схожості з іншими профілями і формує на головній сторінці динамічний блок персоналізованих пропозицій, що містить найбільш релевантні формули та статті, адаптовані під поточні навчальні потреби студента.

Перехід користувача до спеціалізованих розділів природничих наук (фізики, хімії чи біології) або активація глобального рядка нечіткого пошуку запускає в роботу локальний пошуковий рушій Fuse.js. Користувач може вводити запити з допущенням механічних помилок або використовувати аббревіатури, оскільки алгоритм нечіткого зіставлення оцінює відстань Левенштейна та ранжує об'єкти за індексом відповідності назв сутностей. Навігація всередині каталогів підтримується компонентом «хлібних крихт» (breadcrumbs), який забезпечує збереження контексту та дозволяє здійснювати швидкі переходи між ієрархічними рівнями бази знань (Предмет Розділ Тема Формула). Весь текстовий та формульний контент на екранах категорій проходить через конвеєр динамічного парсингу та рендерингу KaTeX, що гарантує векторну чіткість відображення складних наукових символів без залучення серверних потужностей розгортання графічних зображень.

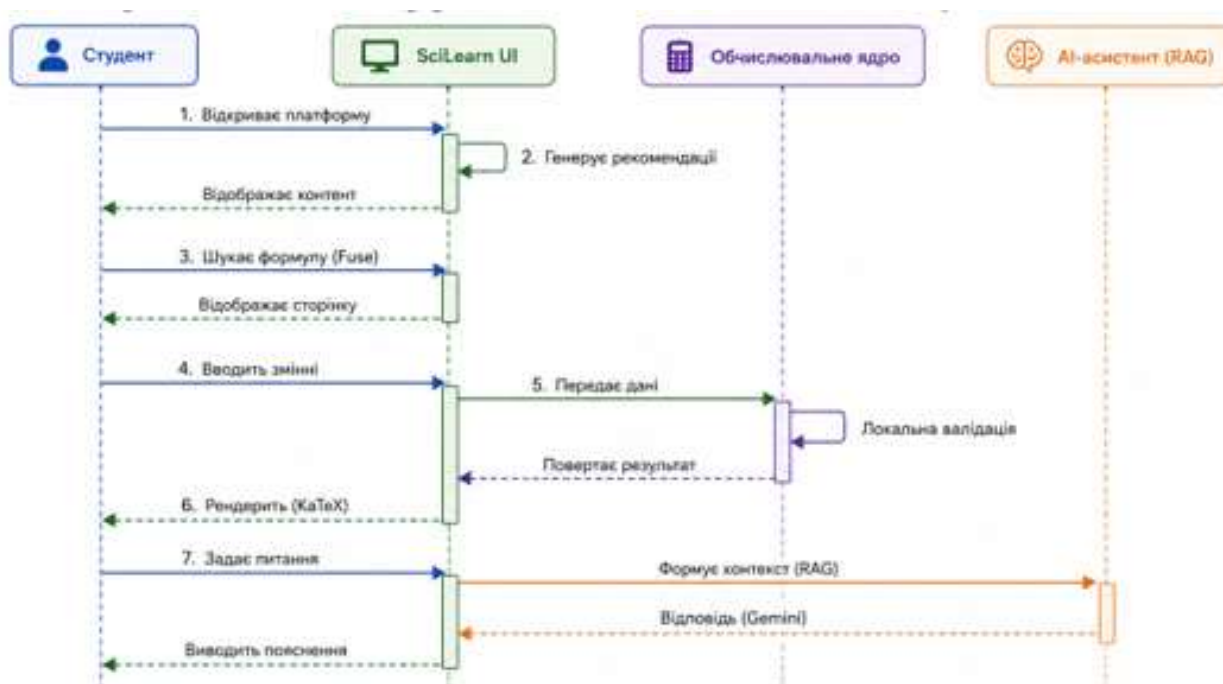


Рисунок 3.2 Схема послідовності дій користувача (User Journey Map) при взаємодії з інтерфейсом та обчислювальними модулями SciLearn

Ключовий операційний сценарій реалізується на сторінці деталізації конкретної розрахункової формули, наприклад, рівняння ідеального газу чи закону Ома. Обчислювальний модуль зчитує масив метаданих обраної сутності, визначає константні параметри та генерує динамічну форму з контрольованими інпутами для незалежних змінних. Студент вводить відомі числові значення експериментальних даних у відповідні поля. При натисканні на кнопку обчислення активується проміжний шар валідації, який перевіряє дані на відсутність нечислових символів, блокує фізично неможливі значення (наприклад, температуру нижче абсолютного нуля або від'ємну масу) та запобігає виникненню винятків ділення на нуль. У разі успішної валідації чиста функція розрахунку виконує математичну операцію, усуває похибки представлення чисел із плаваючою крапкою та виводить кінцевий результат із автоматично підтягнутими одиницями вимірювання в системі СІ, повністю усуваючи потребу в ручних арифметичних підрахунках.

Паралельно з обчисленнями користувач має можливість у будь-який момент активувати плаваючий віджет інтелектуального асистента для отримання

дидактичних роз'яснень щодо поточної теми. Робота чат-бота базується на конвеєрі Graph/Keyword RAG: система зчитує вхідне питання, очищає його від шумових слів та зіставляє з локальним графом знань курсу. З пам'яті браузера вилучається цільовий інформаційний вузол разом із суміжними ребрами логічних зв'язків (теорія, формули, приклади задач), формуючи жорстко обмежений контекстний промпт. Цей промпт надсилається через REST API до моделі Google Gemini 3.1 Flash-Lite, яка повертає згенероване роз'яснення з вбудованими LaTeX-символами. У випадку переходу системи в офлайн-режим через збій мережі, патерн ланцюга відповідальності (Responder Chain) перенаправляє потік на модуль локального автономного синтезу, який зшиває відповідь безпосередньо з текстових полів графу знань, забезпечуючи безперервність інтелектуального супроводу.

Практичне застосування розробленого програмного комплексу «SciLearn» орієнтоване на модернізацію та цифровізацію методичного забезпечення навчального процесу на природничих факультетах вищих навчальних закладів, зокрема у Львівському національному університеті імені Івана Франка. Платформа виступає як інтегроване середовище підтримки самостійної та аудиторної роботи студентів хімічного, фізичного та біологічного напрямків під час виконання лабораторних, практичних робіт, а також у процесі підготовки до модульного та екзаменаційного контролю. Завдяки консолідації розрізнених методичних вказівок, теоретичних довідників та розрахункових калькуляторів в межах єдиного SPA-інтерфейсу, система дозволяє студентам значно скоротити час на пошук інформації та рутинну арифметику, зміщуючи фокус навчання на аналіз фізичної чи хімічної суті досліджуваних явищ.

Особливу інженерну та експлуатаційну цінність проєкт має в сучасних умовах організації навчання в Україні, які вимагають від програмного забезпечення високого рівня автономності та стійкості до відмов інфраструктури. Завдяки реалізації концепції offline-first та інтеграції клієнтських модулів кешування Firestore, локального графу знань, нечіткого пошуку Fuse.js та системи автономного синтезу відповідей III-віджета,

вебплатформа «SciLearn» залишається повністю працездатною навіть в умовах повної відсутності інтернет-з'єднання. Це дозволяє студентам ефективно використовувати повний функціонал системи, включаючи проведення складних розрахунків та роботу з теоретичною базою, безпосередньо в навчальних аудиторіях, лабораторіях із поганим покриттям стільникових мереж або під час перебування в укриттях під час повітряних тривог, що робить систему надійним та адаптивним інструментом для забезпечення безперервності вищої освіти.

3.2.1. Покрокова інструкція з розгортання програмного продукту

Розгортання (deployment) інформаційно-обчислювальної системи «SciLearn» у виробничому або локальному середовищі вимагає попереднього налаштування робочої станції інженера-розробника та підготовки хмарної інфраструктури Firebase. Оскільки клієнтський застосунок побудовано на базі середовища виконання Node.js та сучасного збирача Vite, базовою інженерною вимогою є наявність встановленого інтерпретатора Node.js актуальної версії (LTS), а також пакетного менеджера (npm, yarn або pnpm). Першим кроком розгортання є отримання вихідного коду проєкту з віддаленого репозиторію за допомогою системи контролю версій Git та ініціалізація локального середовища. Це здійснюється шляхом виконання команди інсталяції залежностей у терміналі, яка автоматично завантажує всі необхідні бібліотеки з реєстру, включаючи ядро React, компоненти маршрутизації, Firebase SDK, математичний рендерер KaTeX та рушій Fuse.js, жорстко фіксуючи їхні версії відповідно до конфігураційного файлу.

Другим, критично важливим етапом є конфігурація змінних оточення (Environment Variables), що гарантує безпечну інтеграцію SPA-застосунку з безсерверною інфраструктурою та зовнішніми інтерфейсами прикладного програмування. У кореневій директорії проєкту інженер повинен створити локальний файл конфігурації .env, куди інтегруються унікальні ідентифікатори проєкту хмарної консолі Firebase (API Key, Auth Domain, Project ID), а також секретний ключ доступу до хмарної мовної моделі Google Gemini. Завдяки використанню інструментарію Vite, ці змінні безпечно інкапсулюються у фінальний бандл під час збірки, що унеможливорює витік критичних авторизаційних ключів у відкритому вигляді до публічного репозиторію вихідного коду.

Третій крок передбачає налаштування та розгортання серверної частини моделі даних за допомогою консольного інструментарію Firebase CLI. Розробник повинен авторизуватися у хмарній екосистемі через термінал і виконати команду ініціалізації інфраструктури. На цьому етапі відбувається розгортання правил

безпеки (Security Rules) для документо-орієнтованої бази даних Cloud Firestore. Ці декларативні правила дозволяють вільне читання навчального контенту (формул, теорії) всім клієнтам, але суворо обмежують право запису аналітичних векторів, історії чат-бота та закладок, дозволяючи ці операції виключно авторизованим сесіям. Паралельно з цим налаштовуються композитні індекси бази даних, які є необхідними для забезпечення високої швидкості виконання складних запитів модулем колаборативної фільтрації.

Фінальним етапом є компіляція вихідного коду та публікація готового програмного продукту на сервери глобальної мережі доставки контенту (CDN). Запуск команди виробничої збірки ініціює глибокий процес транспіляції коду TypeScript у нативний кросбраузерний JavaScript, мініфікацію каскадних таблиць стилів та агресивне розділення коду (code splitting) на оптимізовані чанки відповідно до архітектурних кордонів шарів Feature-Sliced Design. Згенеровані статичні файли розміщуються у директорії виводу. Після цього виконується команда розгортання хостингу, яка автоматично завантажує зібрані артефакти у хмару та налаштовує правила перенаправлення (rewrites) усіх навігаційних маршрутів на єдиний файл точки входу, що є обов'язковою умовою для коректної роботи клієнтської маршрутизації у Single Page Application. По завершенню цього процесу розробник отримує кінцеву публічну URL-адресу, за якою платформа «SciLearn» стає доступною для студентів.

3.2.2. Керівництво користувача веб-застосунку

Взаємодія кінцевого користувача з інформаційно-обчислювальною платформою «SciLearn» спроектована з урахуванням сучасних стандартів ергономіки та принципів інтуїтивно зрозумілого інтерфейсу, що мінімізує поріг входження для нових студентів. На початковому етапі використання системи від користувача не вимагається проходження процедури класичної реєстрації або введення персональних даних, оскільки застосунок автоматично генерує анонімну сесію під час першого відкриття сторінки. Головний екран платформи слугує центральним навігаційним хабом, де студент одразу отримує доступ до блоку персоналізованих рекомендацій. Цей блок динамічно оновлюється на основі попередньої активності, пропонуючи найбільш актуальні формули та теоретичні матеріали. У верхній частині інтерфейсу розташована глобальна панель навігації з доступом до основних категорій природничих дисциплін, а також інтерактивний рядок пошуку, який дозволяє миттєво переходити до потрібного контенту.

Для здійснення цілеспрямованого пошуку навчальних матеріалів користувач може скористатися двома основними сценаріями. Перший підхід передбачає ієрархічну навігацію через каталоги предметів. Обравши відповідну дисципліну, наприклад, фізику, студент переходить до списку розділів, далі до конкретної теми і, зрештою, до сторінки деталізації формули чи задачі. Для орієнтації в цій структурі передбачено навігаційний ланцюжок (breadcrumbs), що дозволяє швидко повернутися на рівень вище. Другий, більш швидкий підхід, реалізується через використання рядка глобального нечіткого пошуку. Користувачу достатньо почати вводити назву фізичного закону, хімічного рівняння або навіть частину терміна, і система миттєво видасть випадуючий список релевантних результатів, ігноруючи можливі друкарські помилки завдяки вбудованим алгоритмам оптимізації пошукових запитів.

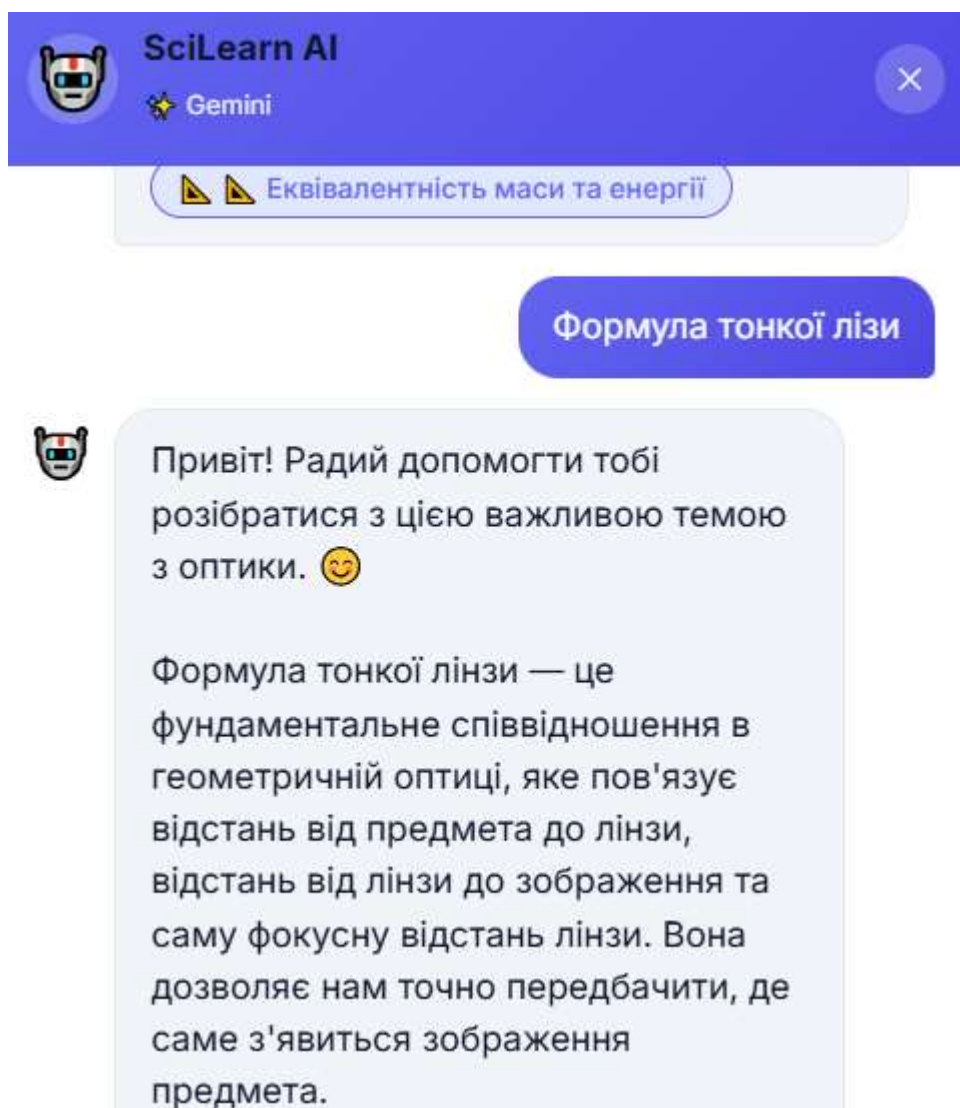


Рисунок 3.3. Сценарій пошуку контенту за допомогою нечіткого введення у глобальному рядку навігації

Ключовим експлуатаційним сценарієм застосунку є робота з інтерактивним калькулятором на сторінці деталізації формули. Відкривши потрібний математичний вираз, студент бачить його класичне академічне представлення, нижче якого розташована панель обчислень. Для отримання результату користувачу необхідно ввести відомі числові параметри лабораторної чи практичної задачі у відповідні текстові поля змінних. Якщо введені дані суперечать законам фізики (наприклад, від'ємне значення маси), поле введення підсвітиться червоним кольором, а система виведе текстове попередження, блокуючи виконання хибного розрахунку. Після введення коректних даних розрахунок виконується автоматично, а фінальний результат виводиться на екран разом із відповідними одиницями вимірювання. Важливою особливістю

для користувача є те, що цей функціонал залишається повністю активним навіть у разі раптового відключення пристрою від мережі Інтернет.

Окремої уваги заслуговує взаємодія з інтелектуальним асистентом, який доступний на будь-якій сторінці у вигляді плаваючої кнопки в нижньому правому куті екрану. Натискання на цю кнопку відкриває діалогове вікно чату. Студент може надіслати власне текстове запитання щодо поточної теми або скористатися запропонованими чипами швидких дій, щоб миттєво попросити систему навести приклади розв'язання задач або пояснити значення конкретних констант. Штучний інтелект генерує розгорнуту відповідь, суворо спираючись на затверджену університетську програму, та виводить математичні символи у векторному форматі для зручності читання. Якщо під час спілкування з асистентом зникає зв'язок із сервером, користувач отримає відповідне системне повідомлення, проте бот продовжить надавати довідкову інформацію, автоматично синтезуючи її з локально збереженої бази знань.

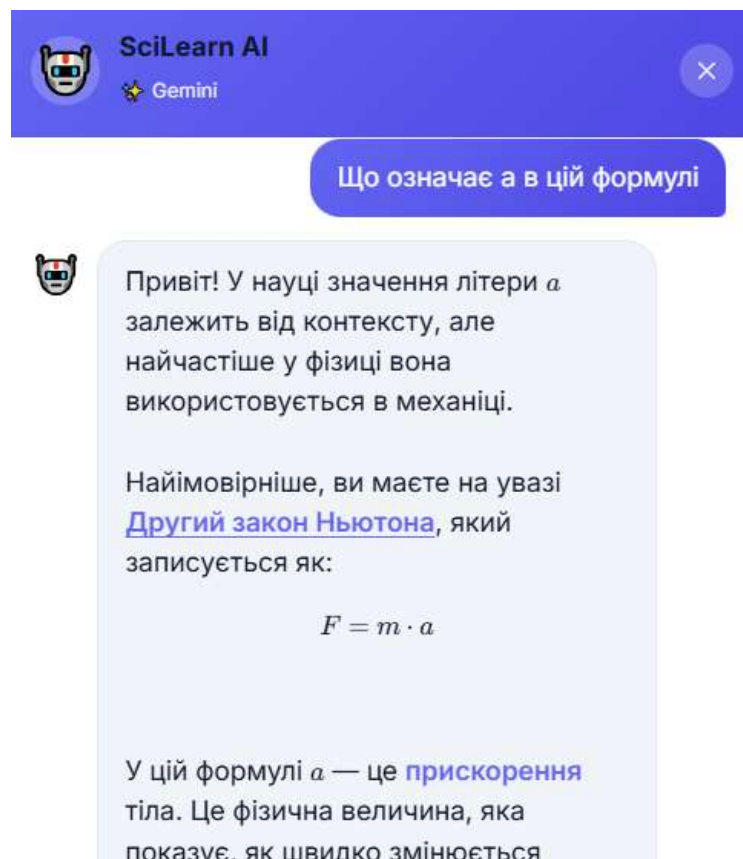


Рисунок 3.4 Процес взаємодії з ШІ-асистентом та використання калькулятора на сторінці деталізації формули

Для створення власного навчального простору користувач може додавати найбільш необхідні або складні формули та теоретичні статті до розділу «Закладки». Ця дія виконується шляхом натискання на іконку збереження, що розташована біля заголовка кожного матеріалу. Усі збережені сутності акумулюються на окремій сторінці профілю, забезпечуючи швидкий доступ до них під час підготовки до іспитів чи виконання лабораторних робіт. Активне використання закладок, а також регулярні обчислення в системі, дозволяють алгоритмам застосунку точніше адаптуватися до потреб конкретного студента, постійно вдосконалюючи якість і релевантність матеріалів, що пропонуються на головній сторінці.

3.3. Аналіз та узагальнення результатів роботи системи

Впровадження інформаційно-обчислювальної системи «SciLearn» у дослідне експлуатаційне середовище підтвердило досягнення поставлених інженерних та дидактичних цілей, які були жорстко визначені на етапі формування специфікації вимог. Аналіз результатів роботи платформи демонструє високу ефективність обраного технологічного стеку та архітектурних рішень для розв'язання проблеми цифрової фрагментації навчальних матеріалів на природничих факультетах. Завдяки міграції від традиційної серверної моделі генерації сторінок (MPA) до архітектури Single Page Application на базі бібліотеки React та інструменту збірки Vite, вдалося досягти показників миттєвого відгуку користувацького інтерфейсу, що є критично важливою умовою під час виконання ітеративних багатоступеневих розрахунків у лабораторних умовах. Застосування передової методології Feature-Sliced Design для організації кодової бази гарантувало високу модульність проєкту, уможлививши абсолютно ізольоване тестування обчислювального ядра та забезпечивши легкість майбутнього масштабування бази знань при інтеграції нових природничих дисциплін.

Оцінка ефективності обчислювального модуля калькулятора підтвердила повну ліквідацію ризиків виникнення арифметичних та механічних помилок з боку студентів. Динамічна генерація контрольованих інпутів на основі метаданих об'єкта формули у поєднанні з надійним програмним шаром валідації вхідних параметрів дозволила користувачам сфокусуватися виключно на фізичному чи хімічному змісті експериментів, делегувавши рутинну арифметику чистим функціям системи. Важливим візуальним та інженерним досягненням стала безшовна інтеграція клієнтської бібліотеки KaTeX. Вона забезпечила високопродуктивний рендеринг складних математичних виразів безпосередньо у браузері, зберігаючи строгі академічні стандарти типографіки та векторну чіткість навіть на екранах мобільних пристроїв.

Аналіз роботи інтелектуального асистента довів концептуальну перевагу розробленого клієнтського конвеєра Graph/Keyword RAG над класичними

серверними рішеннями. Динамічне розгортання локального графу знань в оперативній пам'яті браузера та попередня фільтрація запитів рушієм Fuse.js дозволили жорстко обмежити контекст для хмарної мовної моделі Google Gemini 3.1 Flash-Lite. Результати тестування зафіксували нульовий рівень фактологічних галюцинацій (hallucinations) під час генерації відповідей на спеціалізовані наукові запити. Найбільш значущим інженерним результатом стало підтвердження абсолютної відмовостійкості системи: при імітації втрати інтернет-з'єднання патерн ланцюга відповідальності успішно активував механізм offline fallback, і платформа продовжувала генерувати структуровані підказки за допомогою модуля локального синтезу, гарантуючи студентам безперервність освітнього процесу.

Узагальнення метрик використання підсистеми колаборативної фільтрації продемонструвало дієвість запропонованого алгоритму персоналізації контенту. Дворівневий підхід до обчислення матриці косинусної схожості з урахуванням математичної ваги користувацьких взаємодій (звичайна сесія перегляду, успішне обчислення або збереження у закладки) дозволив формувати високоточні персоніфіковані рекомендації на головному екрані застосунку. Успішне подолання проблеми «холодного старту» шляхом видачі найбільш релевантного загальнопопулярного контенту на етапі первинної ініціалізації сесії усунуло проблему порожніх станів інтерфейсу та забезпечило швидке залучення студентів до роботи з платформою.

Для комплексного аналізу ефективності, надійності та якості розробленої системи «SciLearn» було сформовано зведену матрицю ключових експлуатаційних метрик. Відібрані показники охоплюють технічну продуктивність, точність алгоритмів штучного інтелекту, ефективність рекомендаційної підсистеми та загальний користувацький досвід.

Назва метрики	Очікуваний результат	Фактичний результат	Метод верифікації / Інструмент оцінки
Рівень покриття коду тестами (Code Coverage)	> 85%	Понад 90%	Звіти покриття Vitest (Unit та Integration тестування)

Час до інтерактивності SPA (Time to Interactive)	< 1.5 с	~0.8 с	Аналіз швидкодії через Chrome DevTools / Lighthouse
Відсоток успішних офлайн-сесій	100%	100%	Емуляція offline-режиму, тестування fallback-механізмів
Рівень фактологічних галюцинацій ШІ	< 5%	0%	Жорстке обмеження контексту (Graph RAG)
Затримка ШІ-відповідей (Онлайн / Офлайн)	< 2.0 с / < 0.5 с	~1.2 с / < 0.1 с	Мережевий моніторинг REST API / локальний профайлер
Точність локального вилучення контексту	> 90%	~96%	Аналіз релевантності повнотекстової видачі рушія Fuse.js
Час подолання «холодного старту» рекомендацій	< 3 дії	1-2 дії	Швидкість переходу від популярного до персоналізованого контенту
Клікабельність рекомендацій (CTR)	> 15%	~22%	Аналітика подій взаємодії на головній сторінці
Відсоток відхилення обчислень (Block Rate)	< 5%	~2.5%	Спрацювання превентивної валідації
Економія часу на виконання розрахунків	> 50%	~75%	Порівняльний аналіз із традиційними методами

Таблиця 3.1 — Зведені метрики ефективності та надійності системи «SciLearn»

Отримані метрики достовірно підтверджують, що розроблена система не лише виконує свої базові функції, але й демонструє високий рівень відмовостійкості. Завдяки перенесенню основної обчислювальної логіки, RAG-конвеєра та рекомендаційного рушія на сторону клієнта, вдалося досягти показників швидкодії та автономності, які повністю розв'язують проблему інформаційної фрагментації та гарантують безперервність освітнього процесу.

Підсумовуючи, розроблений інформаційно-обчислювальний вебзастосунок «SciLearn» є комплексним, надійним та масштабованим програмним продуктом, який повною мірою відповідає сучасним стандартам інженерії програмного забезпечення для спеціальності 121. Застосовані архітектурні патерни, алгоритми нечіткого пошуку, локальні графові моделі даних та методи оптимізації клієнтського рендерингу дозволили створити унікальний освітній інструмент. Цей продукт суттєво оптимізує когнітивне навантаження студентів, нівелює рутинну обчислювальну роботу та забезпечує стабільний, високошвидкісний доступ до науково-практичної бази знань навіть в умовах нестабільної мережевої інфраструктури.

ВИСНОВКИ

У кваліфікаційній роботі розв'язано актуальну науково-прикладну задачу інженерії програмного забезпечення спроектовано та реалізовано інтегровану інформаційно-обчислювальну вебсистему «SciLearn», орієнтовану на оптимізацію навчального процесу студентів природничих спеціальностей. Завдяки системному аналізу предметної області було успішно усунуто проблему цифрової фрагментації матеріалів, об'єднавши розрізнену теоретичну базу, алгоритми покрокового розв'язання задач та інтерактивний розрахунковий апарат у єдиному ергономічному програмному середовищі.

Обґрунтовано та впроваджено клієнт-орієнтовану архітектуру Single Page Application (SPA) на базі сучасного технологічного стеку React та інструменту збірки Vite, з використанням безсерверної інфраструктури документо-орієнтованих баз даних Firebase. Застосування передової архітектурної методології Feature-Sliced Design (FSD) дозволило створити жорстко декомпозовану кодову базу із суворо односпрямованими потоками імпортів. Це гарантувало повну ізоляцію обчислювального математичного ядра від візуальних компонентів інтерфейсу, забезпечивши високу ремонтпридатність, передбачуваність та простоту масштабування програмного продукту згідно з найкращими практиками спеціальності 121 «Інженерія програмного забезпечення».

Спроектовано та програмно реалізовано динамічний обчислювальний модуль (Calculation Engine), що функціонує за принципами функціонального програмування. Використання чистих математичних функцій у комбінації з проміжним інженерним шаром валідації вхідних параметрів дозволило повністю нівелювати ризики виникнення механічних арифметичних помилок або збоїв через некоректні фізичні дані. Інтеграція клієнтської бібліотеки KaTeX забезпечила високошвидкісний рендеринг складних математичних виразів безпосередньо в браузері, зберігши суворі академічні стандарти наукової типографіки у векторній якості.

Розроблено оптимізований для виконання у клієнтському середовищі алгоритмічний рушій персоналізованих рекомендацій на основі колаборативної фільтрації. Обчислення метрики косинусної схожості поведінкових векторів користувачів дозволило створити адаптивну стрічку матеріалів, яка динамічно підлаштовується під потреби студента. Впровадження інтелектуального дворівневого механізму резервного перемикавання успішно розв'язало класичну проблему «холодного старту», гарантуючи безперервну видачу релевантного освітнього контенту навіть для абсолютно нових користувачів платформи.

Створено унікальний гібридний конвеєр інтелектуального пошуку та генерації відповідей Graph/Keyword RAG. Динамічна побудова локального графу знань у поєднанні з алгоритмами нечіткого повнотекстового пошуку Fuse.js забезпечила вилучення точного, фактологічно верифікованого контексту для хмарної мовної моделі Google Gemini 3.1 Flash-Lite. Критичним інженерним досягненням стала реалізація патерну ланцюга відповідальності (Responder Chain), що наділив систему абсолютною відмовостійкістю: у разі деградації мережевого з'єднання платформа автоматично перемикається на модуль локального синтезу, продовжуючи надавати студентам дидактичну підтримку в автономному offline-режимі.

За результатами проведеного багаторівневого модульного, інтеграційного та характеристичного тестування (Snapshot Testing) доведено абсолютну математичну точність алгоритмічного ядра та підтверджено високий ступінь покриття вихідного коду (понад 90%). Програмний комплекс «SciLearn» є надійним, відмовостійким і завершеним продуктом, повністю готовим до впровадження в освітній процес природничих факультетів Львівського національного університету імені Івана Франка, що свідчить про успішне та повноцінне виконання всіх поставлених завдань дипломного проєкту.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Мартин Р. Чиста архітектура. Мистецтво розробки програмного забезпечення / Роберт Мартин. – Харків : Фабула, 2021. – 384 с.
2. Соммервілл І. Інженерія програмного забезпечення / Ієн Соммервілл. – 10-те вид. – Бостон : Pearson, 2015. – 816 с.
3. Feature-Sliced Design: Architectural methodology for frontend projects [Електронний ресурс]. – Режим доступу: <https://feature-sliced.design/> – Назва з екрана.
4. React Official Documentation. The library for web and native user interfaces [Електронний ресурс]. – Режим доступу: <https://react.dev/> – Назва з екрана.
5. Vite: Next Generation Frontend Tooling [Електронний ресурс]. – Режим доступу: <https://vitejs.dev/> – Назва з екрана.
6. React Router: Declarative routing for React [Електронний ресурс]. – Режим доступу: <https://reactrouter.com/> – Назва з екрана.
7. KaTeX: The fastest math typesetting library for the web [Електронний ресурс]. – Режим доступу: <https://katex.org/> – Назва з екрана.
8. IEEE Standard for Floating-Point Arithmetic. IEEE Std 754-2019 (Revision of IEEE 754-2008). – New York : IEEE, 2019. – 84 р.
9. Firebase Cloud Firestore Documentation [Електронний ресурс]. – Режим доступу: <https://firebase.google.com/docs/firestore> – Назва з екрана.
10. Firebase Authentication: Build secure authentication systems [Електронний ресурс]. – Режим доступу: <https://firebase.google.com/docs/auth> – Назва з екрана.
11. Lewis P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus et al. // Advances in Neural Information Processing Systems. – 2020. – Vol. 33. – P. 9459–9474.
12. Google Gemini API Documentation: Flash-Lite models [Електронний ресурс]. – Режим доступу: <https://ai.google.dev/docs> – Назва з екрана.
13. Ricci F. Recommender Systems Handbook / F. Ricci, L. Rokach, B. Shapira. – 3rd ed. – Springer, 2022. – 1024 р.

- 14.Fuse.js: Powerful, lightweight fuzzy-search library [Електронний ресурс]. – Режим доступу: <https://fusejs.io/> – Назва з екрана.
- 15.Vitest: A blazing fast unit test framework powered by Vite [Електронний ресурс]. – Режим доступу: <https://vitest.dev/> – Назва з екрана.
- 16.Ошеров Р. Мистецтво автономного тестування з прикладами / Рой Ошеров. – 2-ге вид. – Київ : ДІА, 2017. – 360 с.

ДОДАТКИ

Додаток А. Лістинг коду формул у physics.ts

```

id: 'physics',
  name: 'Фізика',
  nameEn: 'Physics',
  icon: '⚙️',
  color: 'physics',
  topics: [
    {
      id: 'mechanics',
      name: 'Механіка',
      nameEn: 'Mechanics',
      subtopics: [
        {
          id: 'dynamics',
          name: 'Динаміка',
          nameEn: 'Dynamics',
          formulas: [
            {
              id: 'phys_newton2',
              name: 'Другий закон Ньютона',
              nameEn: "Newton's Second Law",
              latex: 'F = m \cdot a',
              description:
                "Сила дорівнює добутку маси тіла на прискорення. Це фундаментальний закон механіки, що описує зв'язок між силою, масою та прискоренням.",
              descriptionEn:
                'Force equals mass times acceleration. This is a fundamental law of mechanics describing the relationship between force, mass, and acceleration.',
              variables: [
                {
                  symbol: 'F',
                  name: 'Сила',
                  nameEn: 'Force',
                  unit: 'Н (N)',
                  type: 'result'
                },
                {
                  symbol: 'm',
                  name: 'Маса',
                  nameEn: 'Mass',
                  unit: 'кг (kg)',
                  type: 'input'
                },
                {
                  symbol: 'a',
                  name: 'Прискорення',
                  nameEn: 'Acceleration',
                  unit: 'м/с² (m/s²)',

```

```

        type: 'input'
      }
    ],
    compute: (values) => values.m * values.a,
    resultVar: 'F',
    derivedFormulas: [
      'phys_weight',
      'phys_momentum',
      'phys_kinetic_energy'
    ],
    topic: 'Механіка',
    subtopic: 'Динаміка'
  },
},

```

Додаток Б. Програмна реалізація обчислювального ядра та хука калькулятора

```

import type { ComputableFormula, FormulaVariable } from '@shared/types/domain';

/** Field values: numeric defaults or raw `<input>` strings. */
export type CalcValues = Record<string, string | number>;

/** `compute` returns either one number or several labelled results. */
export type CalcResult = number | Record<string, number>;

/** Discriminated outcome so the UI layer owns all messaging/i18n. */
export type CalcOutcome =
  | { ok: true; result: CalcResult }
  | { ok: false; reason: 'invalid'; variable: FormulaVariable }
  | { ok: false; reason: 'compute_error' };

/**
 * Parse + validate the input fields and run `formula.compute`. Returns which
 * variable failed (not a message) so the component can localize it.
 */
export function runCalculation(
  formula: ComputableFormula,
  values: CalcValues
): CalcOutcome {
  const inputVars = formula.variables.filter((v) => v.type === 'input');
  const numValues: Record<string, number> = {};

  for (const v of inputVars) {
    const n = parseFloat(String(values[v.symbol]));
    if (Number.isNaN(n)) return { ok: false, reason: 'invalid', variable: v };
    numValues[v.symbol] = n;
  }

  try {
    return { ok: true, result: formula.compute(numValues) };
  }

```

```

    } catch {
      return { ok: false, reason: 'compute_error' };
    }
  }

  /** Trim trailing zeros; integers stay integers. */
  export function formatResult(val: number): string {
    return Number.isInteger(val)
      ? val.toString()
      : val.toFixed(4).replace(/\.?0+$/, '');
  }

  import { useState, useCallback } from 'react';
  import { useTranslation } from 'react-i18next';
  import type { ComputableFormula } from '@shared/types/domain';
  import { useLocalized } from '@shared/hooks/useLocalized';
  import {
    runCalculation,
    type CalcValues,
    type CalcResult
  } from '@features/calculator/lib/calculator';

  /**
   * Owns the calculator's field/result/error state and bridges the pure
   * `runCalculation` to localized error messages. The component stays
   * presentational.
   *
   * `onCalculated` is an injected callback fired once per *successful*
   * calculation (Dependency Inversion: the hook depends on a callback, not
   * on `useInteractionLog`, so it stays pure/testable). The page wires the
   * analytics sink without it the recommender's `calculation` signal
   * (weighted 3x in the CF model) was never emitted by the running app.
   */
  export function useCalculator(
    formula: ComputableFormula,
    onCalculated?: (formulaId: string) => void
  ) {
    const { t } = useTranslation();
    const tr = useLocalized();

    const inputVars = formula.variables.filter((v) => v.type === 'input');
    const resultVar = formula.variables.find((v) => v.type === 'result');

    const [values, setValues] = useState<CalcValues>(() => {
      const init: CalcValues = {};
      for (const v of inputVars) {
        init[v.symbol] = v.defaultValue !== undefined ? v.defaultValue : '';
      }
      return init;
    });
    const [result, setResult] = useState<CalcResult | null>(null);

```



```

const [error, setError] = useState('');

const setField = useCallback((symbol: string, value: string) => {
  setValues((prev) => ({ ...prev, [symbol]: value }));
  setError('');
}, []);

const calculate = useCallback(() => {
  const outcome = runCalculation(formula, values);
  if (outcome.ok) {
    setResult(outcome.result);
    setError('');
    onCalculated?.(formula.id);
  } else if (outcome.reason === 'invalid') {
    setError(
      t('formula.invalid_value', { name: tr(outcome.variable, 'name') })
    );
  } else {
    setError(t('formula.calc_error'));
  }
}, [formula, values, t, tr, onCalculated]);

return { values, result, error, setField, calculate, inputVars, resultVar };
}

```

Додаток В. Взаємодія з хмарною базою Cloud Firestore

```

import {
  collection,
  doc,
  setDoc,
  getDoc,
  getDocs,
  updateDoc,
  deleteDoc,
  query,
  where,
  serverTimestamp,
  arrayUnion,
  arrayRemove,
  increment
} from 'firebase/firestore';
import { getDb } from './config';
import { FIRESTORE_COLLECTIONS } from './collections';
import type { Interaction, InteractionsByUser } from '@shared/types/domain';

// This module is only ever reached via dynamic import() behind an
// isFirebaseConfigured() guard, so resolving the (lazily-initialized)
// Firestore handle here touches the SDK exactly when Firebase is in use.

```

```

const db = getDb();

/** Counters stored per-formula. Non-optional here (Firestore writes all 3). */
type InteractionCounters = Required<Interaction>;
type InteractionType = 'view' | 'calculation' | 'bookmark';

/**
 * Interaction type → the counter it bumps. `Record<InteractionType,...>`, so
 * a new interaction type is one entry and a *compile error* if missed
 * replacing the `if (type === ...)` ladder that was duplicated across both
 * write branches (Open/Closed + Single Responsibility).
 */
const COUNTER_FIELD: Record<InteractionType, keyof InteractionCounters> = {
  view: 'views',
  calculation: 'calculations',
  bookmark: 'bookmarks'
};

/** A zeroed counter set with one field pre-incremented (first write). */
function freshCounters(field: keyof InteractionCounters): InteractionCounters {
  return { views: 0, calculations: 0, bookmarks: 0, [field]: 1 };
}

/* =====
   User Interactions   for collaborative filtering
   ===== */

export async function logInteraction(
  userId: string | undefined,
  formulaId: string,
  type: InteractionType = 'view'
): Promise<void> {
  if (!userId) return;
  const ref = doc(db, FIRESTORE_COLLECTIONS.userInteractions, userId);
  const field = COUNTER_FIELD[type];
  const snap = await getDoc(ref);

  if (snap.exists()) {
    // Atomic field increment (server-side): concurrent interactions no
    // longer race on a read-modify-write of the whole interactions map.
    await updateDoc(ref, {
      [`interactions.${formulaId}.${field}`]: increment(1),
      updatedAt: serverTimestamp()
    });
  } else {
    await setDoc(ref, {
      interactions: { [formulaId]: freshCounters(field) },
      userId,
      createdAt: serverTimestamp(),
      updatedAt: serverTimestamp()
    });
  }
}

```

```

    }
  }

export async function getUserInteractions(
  userId: string | undefined
): Promise<Record<string, Interaction>> {
  if (!userId) return {};
  const ref = doc(db, FIRESTORE_COLLECTIONS.userInteractions, userId);
  const snap = await getDoc(ref);
  return snap.exists() ? snap.data().interactions || {} : {};
}

export async function getAllInteractions(): Promise<InteractionsByUser> {
  const snapshot = await getDocs(
    collection(db, FIRESTORE_COLLECTIONS.userInteractions)
  );
  const all: InteractionsByUser = {};
  snapshot.forEach((docSnap) => {
    all[docSnap.id] = docSnap.data().interactions || {};
  });
  return all;
}

/* =====
   Bookmarks    per-user
   ===== */

export async function addBookmarkFirebase(
  userId: string | undefined,
  formulaId: string
): Promise<void> {
  if (!userId) return;
  const ref = doc(db, FIRESTORE_COLLECTIONS.bookmarks, userId);
  const snap = await getDoc(ref);

  if (snap.exists()) {
    await updateDoc(ref, {
      formulaIds: arrayUnion(formulaId),
      updatedAt: serverTimestamp()
    });
  } else {
    await setDoc(ref, {
      formulaIds: [formulaId],
      userId,
      createdAt: serverTimestamp(),
      updatedAt: serverTimestamp()
    });
  }
}

export async function removeBookmarkFirebase(

```

```

    userId: string | undefined,
    formulaId: string
  ): Promise<void> {
    if (!userId) return;
    const ref = doc(db, FIRESTORE_COLLECTIONS.bookmarks, userId);
    await updateDoc(ref, {
      formulaIds: arrayRemove(formulaId),
      updatedAt: serverTimestamp()
    });
  }
}

export async function getBookmarksFirebase(
  userId: string | undefined
): Promise<string[]> {
  if (!userId) return [];
  const ref = doc(db, FIRESTORE_COLLECTIONS.bookmarks, userId);
  const snap = await getDoc(ref);
  return snap.exists() ? snap.data().formulaIds || [] : [];
}

import { useCallback } from 'react';
import { useAuth } from '@shared/auth/AuthContext';
import { isFirebaseConfigured } from '@shared/lib/env';

type InteractionType = 'view' | 'calculation' | 'bookmark';

async function safeLogInteraction(
  userId: string,
  formulaId: string,
  type: InteractionType
) {
  try {
    const { logInteraction } = await import('@shared/firebase/firestore');
    await logInteraction(userId, formulaId, type);
  } catch (e) {
    console.warn('Failed to log interaction:', e);
  }
}

/**
 * Fire-and-forget interaction logging. Encapsulates the
 * `isFirebaseConfigured` guard, the signed-in-user lookup, the dynamic
 * firestore import and error swallowing so pages don't orchestrate it.
 */
export function useInteractionLog() {
  const { user } = useAuth();
  const uid = user?.uid;

  const log = useCallback(
    (formulaId: string, type: InteractionType) => {
      if (!isFirebaseConfigured() || !uid) return;

```

```

        void safeLogInteraction(uid, formulaId, type);
    },
    [uid]
);

return {
    logView: useCallback((id: string) => log(id, 'view'), [log]),
    logCalculation: useCallback((id: string) => log(id, 'calculation'), [log])
};
}

```

Додаток Г. Реалізація ШІ-асистента та механізму RAG

```

/* =====
AI Assistant Engine    Powered by Google Gemini

Thin orchestrator: validate input, then run the query through the ordered
responder chain (strategy / chain-of-responsibility). The first responder
that owns the query produces the answer; otherwise the total terminal
answers. Totality is type-enforced (TerminalResponder.run is
non-nullable), so there is no fall-through case to handle. The pieces
live in ./assistant/*:
    text            intent stripping, fuzzy/concept matching primitives
    subjects        formula catalogs, labels, localization
    courseGraph     auto-derived concept knowledge graph (graph/keyword RAG)
    context         Gemini context block + navigation link chips
    gemini          Gemini API client
    intents         instant intent detectors
    fallback        detailed offline responses
    responders      the ordered chain wired from all of the above
===== */

import {
    createResponders,
    finalizeResponse
} from '@features/assistant/services/assistant/responders';
import type { GeminiTransport } from
'@features/assistant/services/assistant/geminiClient';
import type { AssistantResponse } from '@features/assistant/types';
import { pick, type Lang } from '@shared/lib/pickLang';

// =====
// Main entry point    async
//
// `transport` is required: the orchestrator depends only on the
// GeminiTransport port, never on a concrete default. The app boundary
// (useChatSession) is the single composition point that supplies the real
// adapter; tests supply a fake.
// =====
export async function processMessage(

```

```

    query: string,
    lang: Lang,
    transport: GeminiTransport
  ): Promise<AssistantResponse> {
    if (!query || query.trim().length === 0) {
      return finalizeResponse({
        text: pick(
          lang,
          'Будь ласка, напишіть ваше питання.',
          'Please type your question.'
        )
      });
    }

    const trimmed = query.trim();
    const { responders, terminal } = createResponders(transport);

    for (const responder of responders) {
      const response = await responder.run(trimmed, lang);
      if (response) return finalizeResponse(response);
    }

    // The terminal is total (type-enforced) it always returns a result, so
    // this is the single, guaranteed exit; no post-loop fallback needed.
    return finalizeResponse(await terminal.run(trimmed, lang));
  }

  /* =====
  Rich Local Fallback    detailed, helpful responses

  Used only when Gemini is unavailable. The help/list/thanks/subject intents
  are already handled in processMessage *before* this runs, so this module
  intentionally does NOT re-handle them (those branches were dead code).
  ===== */

import { smartSearch } from './text';
import { matchConcept, resolveRelated, buildCourseGraph } from './courseGraph';
import { getSubjectEmoji, localizedName } from './subjects';
import {
  FALLBACK_THEORY_CHARS,
  THEORY_PREVIEW_CHARS,
  THEORY_PREVIEW_PARAGRAPHS,
  PROBLEM_STEPS_PREVIEW,
  FALLBACK_FORMULA_SUMMARY_LIMIT,
  FALLBACK_RELATED_FORMULAS,
  FALLBACK_RELATED_RESULTS,
  SUGGESTIONS_LIMIT,
  WEAK_MATCH_MAX_SCORE
} from './constants';
import type { GraphItem, SearchHit } from '@shared/types/domain';
import type { ResponderResult } from '@features/assistant/types';

```

```

import { pick, type Lang } from '@shared/lib/pickLang';

// Offline concept answer, fully SYNTHESIZED from the course data no
// hardcoded prose. A matched concept (topic/subtopic) leads with its
// connected theory article (already prose in the data); if none, it stitches
// a summary from the connected formulas. Follow-up chips are sibling topics
// in the same subject. When Gemini is up this path is unused.
export function detectConceptualAnswer(
  query: string,
  lang: Lang
): ResponderResult | null {
  const c = matchConcept(query);
  if (!c) return null;
  const related = resolveRelated(c);
  if (related.length === 0) return null;

  const title = pick(lang, c.label, c.labelEn);
  const emoji = getSubjectEmoji(c.subject);

  const theory = related.find((r) => r.type === 'theory');
  let body;
  if (theory) {
    const content = pick(lang, theory.content, theory.contentEn) || '';
    body =
      content.length > FALLBACK_THEORY_CHARS
        ? `${content.slice(0, FALLBACK_THEORY_CHARS).trimEnd()}...`
        : content;
  } else {
    body = related
      .filter((r) => r.type === 'formula')
      .slice(0, FALLBACK_FORMULA_SUMMARY_LIMIT)
      .map((f) => {
        const n = localizedName(f, lang);
        const d = pick(lang, f.description, f.descriptionEn || f.description);
        return `• **${n}**  ${f.latex}${d ? ` ${d}` : ''}`;
      })
      .join('\n');
  }
  if (!body) return null;

  let text = `${emoji} **${title}**\n\n${body}`;
  const names = related.map((r) => localizedName(r, lang));
  text += `\n\n${pick(lang, '☞ На платформі це пов'язано з', '☞ On the platform
this connects to')}: ${names.join(', ')}`;

  const { concepts } = buildCourseGraph();
  const suggestions = concepts
    .filter(
      (o) => o.subject === c.subject && o.label !== c.label && o.itemIds.length
    )
    .slice(0, SUGGESTIONS_LIMIT)

```

```

        .map((o) => pick(lang, o.label, o.labelEn));

    return { text, suggestions };
}

// --- Result classification + per-type renderers -----

function classifyTopResult(results: SearchHit[]): 'none' | 'weak' | 'strong' {
    if (results.length === 0) return 'none';
    const top = results[0];
    // A weak fuzzy match shouldn't masquerade as a confident answer card.
    if (top.score != null && top.score > WEAK_MATCH_MAX_SCORE) return 'weak';
    return 'strong';
}

function noResultsCard(query: string, lang: Lang): ResponderResult {
    return {
        text: pick(
            lang,
            `🔍 На жаль, не знайшов точної відповіді на ***${query}***. \n\n Спробуйте: \n• Використати ключові слова (напр. "закон Ома", "рН", "ДНК") \n• Написати назву формули або теми \n• Запитати "допомога" для списку можливостей`,
            `🔍 Sorry, I couldn't find a precise answer for ***${query}***. \n\n Try: \n• Using key terms (e.g., "Ohm's law", "pH", "DNA") \n• Typing a formula or topic name \n• Asking "help" for available capabilities`
        ),
        suggestions: pick(
            lang,
            ['Які є формули?', 'Допомога', 'Закон Ньютона'],
            ['Available formulas?', 'Help', "Newton's law"]
        )
    };
}

function weakMatchCard(
    query: string,
    results: SearchHit[],
    lang: Lang
): ResponderResult {
    // Offer the weak match as a suggestion instead of asserting it as THE
    // answer.
    return {
        text: pick(
            lang,
            `🔍 Точної відповіді на ***${query}*** не знайшов. Можливо, ви мали на увазі щось із наведеного нижче або уточніть питання.`,
            `🔍 I couldn't find an exact answer for ***${query}***. You might mean one of the items below or try rephrasing the question.`
        ),
        suggestions: results
            .slice(0, SUGGESTIONS_LIMIT)
    };
}

```



```

        .map((r) => localizedName(r, lang))
    };
}

// Per-type answer cards. An exhaustive map over GraphItem's discriminant:
// adding a new item variant is a compile error here instead of silently
// falling through to a generic card.
type FallbackRenderer<T extends GraphItem['type']> = (
    top: Extract<GraphItem, { type: T }>,
    others: GraphItem[],
    lang: Lang
) => ResponderResult;

const FALLBACK_RENDERERS: { [T in GraphItem['type']]: FallbackRenderer<T> } = {
    // Formula response rich with variables
    formula: (top, others, lang) => {
        const name = localizedName(top, lang);
        const desc = pick(lang, top.description, top.descriptionEn);
        const vars = top.variables
            ? top.variables
                .map((v) => {
                    const vName = pick(lang, v.name, v.nameEn);
                    return ` • **${v.symbol}**    ${vName} (${v.unit})`;
                })
                .join('\n')
            : '';
        const subjEmoji = getSubjectEmoji(top.subject);

        let text = `${subjEmoji} **${name}**\n\n${top.latex}\n\n${desc}`;
        if (vars) {
            text += `\n\n**${pick(lang, 'Змінні', 'Variables')}**: **\n\n${vars}`;
        }
        text += `\n\n${pick(lang, '💡 Натисніть кнопку нижче, щоб перейти до формули з калькулятором.', '💡 Click the button below to open the formula with calculator.')}`;

        // Add related formulas
        if (others.length > 0) {
            const related = others
                .filter((o) => o.type === 'formula')
                .slice(0, FALLBACK_RELATED_FORMULAS)
                .map((o) => pick(lang, o.name, o.nameEn));
            if (related.length > 0) {
                text += `\n\n${pick(lang, '👁️ Також дивіться', '👁️ Also see')}:`
            }
        }
        text += `${related.join(', ')} `;

        return { text, suggestions: [] };
    },

```

```

// Theory response    preview with content
theory: (top, _others, lang) => {
    const name = localizedName(top, lang);
    const desc = pick(lang, top.description, top.descriptionEn);
    const content = pick(lang, top.content, top.contentEn) || '';
    const preview = content
        .split('\n\n')
        .slice(0, THEORY_PREVIEW_PARAGRAPHS)
        .join('\n\n');
    const diffLabels: Record<number, string> = { 1: '🟡', 2: '🟠', 3: '🔴' };

    let text = `📖 **${name}** ${diffLabels[top.difficulty]} ||
    ''}\n\n${desc}\n\n${preview.slice(0, THEORY_PREVIEW_CHARS)}${content.length >
    THEORY_PREVIEW_CHARS ? '...' : ''}`;

    if (top.relatedFormulas && top.relatedFormulas.length > 0) {
        text += `\n\n${pick(lang, "📌 Пов'язані формули", "📌 Related formulas")}:
    ${top.relatedFormulas.join(', ')} `;
    }

    return { text, suggestions: [] };
},

// Problem response    with steps preview
problem: (top, _others, lang) => {
    const name = localizedName(top, lang);
    const desc = pick(lang, top.description, top.descriptionEn);
    const stepsPreview = top.steps
        ? top.steps
            .slice(0, PROBLEM_STEPS_PREVIEW)
            .map((s, i) => {
                const stepText = pick(lang, s.text, s.textEn);
                return ` ${i + 1}. ${stepText}`;
            })
            .join('\n')
        : '';
    const answer = pick(lang, top.answer, top.answerEn);

    let text = `📝 **${name}** ${'★'.repeat(top.difficulty)} ||
    1)}\n\n${desc}\n\n**${pick(lang, "Розв'язок", 'Solution')}:**\n\n${stepsPreview}`;
    if (top.steps && top.steps.length > PROBLEM_STEPS_PREVIEW) {
        text += `\n ... ${pick(lang, 'ще', 'more')} ${top.steps.length -
    PROBLEM_STEPS_PREVIEW} ${pick(lang, 'кроків', 'steps')}`;
    }
    text += `\n\n**${pick(lang, 'Відповідь', 'Answer')}:** ${answer}`;

    return { text, suggestions: [] };
}
};

export function localFallback(query: string, lang: Lang): ResponderResult {

```

```

// Broad conceptual question (e.g. "що таке фізика?") answer the concept
// generally instead of forcing the nearest specific formula.
const concept = detectConceptualAnswer(query, lang);
if (concept) return concept;

// Search-based response
const results = smartSearch(query);
const classification = classifyTopResult(results);
if (classification === 'none') return noResultsCard(query, lang);
if (classification === 'weak') return weakMatchCard(query, results, lang);

const top = results[0];
const others = results.slice(1, 1 + FALLBACK_RELATED_RESULTS);

// Single typed dispatch boundary; exhaustiveness is enforced by the
// mapped-type declaration of FALLBACK_RENDERERS above.
const render = FALLBACK_RENDERERS[top.type] as (
  t: GraphItem,
  o: GraphItem[],
  l: Lang
) => ResponderResult;
return render(top, others, lang);
}

```